

Università degli Studi di Salerno

Tesi di Laurea Triennale in Informatica

ANALISI E SVILUPPO DI UN MODERNO FILESYSTEM CRITTOGRAFICO TRASPARENTE PER LINUX: TCFS

Carlo Alberto Giordano

matr. 0512110442

Relatore:

Prof. Giuseppe Cattaneo

21 Marzo 2024

Abstract

L'obiettivo di questa tesi è analizzare e sviluppare un moderno filesystem crittografico trasparente per il sistema operativo Linux, concentrandosi sul caso di studio di TCFS. Dopo una valutazione critica degli approcci esistenti, si è proceduto alla progettazione e implementazione di TCFS, integrando l'algoritmo di cifratura AES per garantire la sicurezza dei dati. Questa tesi fornisce una panoramica approfondita dell'analisi e dello sviluppo di TCFS come moderno filesystem crittografico, evidenziando i benefici e le sfide associate alla sua implementazione e offrendo spunti per ulteriori ricerche nel campo della sicurezza informatica e dei sistemi operativi.

Sommario

Il presente lavoro di tesi si propone di analizzare e sviluppare un moderno filesystem crittografico trasparente per il sistema operativo Linux, con particolare attenzione al caso di studio di TCFS. Dopo una breve introduzione al tema e alla motivazione del lavoro, vengono esposti gli obiettivi della ricerca e la metodologia adottata.

Il primo capitolo fornisce alcuni cenni storici su TCFS, esaminando il contesto storico nel quale è nato, la sua prima implementazione e i miglioramenti della nuova versione di TCFS. Si descrive in dettaglio il contesto storico e le motivazioni che hanno portato alla riscrittura totale di TCFS.

Il secondo capitolo fornisce una definizione approfondita di TCFS, esaminandone le caratteristiche principali e il funzionamento. Si descrivono in dettaglio l'architettura del sistema, comprensiva del server REST, del key manager e del client, nonché l'interfaccia con FUSE. Vengono discussi i vantaggi e gli svantaggi di TCFS e le sfide ancora aperte.

Il terzo capitolo approfondisce l'algoritmo di cifratura AES, utilizzato da TCFS per garantire la sicurezza dei dati. Si analizzano le caratteristiche di AES, il processo di cifratura e decifratura dei file, nonché l'integrazione di AES con TCFS, compresa la gestione delle chiavi e la verifica dell'integrità dei file.

Nel quarto capitolo si analizza in dettaglio il processo di lettura e scrittura all'interno di TCFS, si scende in dettaglio e si guarda al codice che gestisce le operazioni di lettura e scrittura, insieme alle operazioni di generazione e gestione delle chiavi.

Nel quinto capitolo, vengono valutate le prestazioni e la sicurezza di TCFS attraverso criteri e metodi di valutazione specifici. Si confronta TCFS con altri filesystem crittografici, si identificano possibili attacchi e si discutono le contromisure adottate. Infine, vengono presentati i risultati dell'analisi e le relative discussioni.

Le conclusioni riassumono i risultati ottenuti, delineando i limiti del lavoro svolto e suggerendo possibili sviluppi futuri. Si offrono anche considerazioni personali e critiche sul processo di ricerca e sviluppo.

Il presente documento si propone quindi di offrire una panoramica completa sull'analisi e lo sviluppo di TCFS come moderno filesystem crittografico trasparente per Linux, con riferimenti agli aspetti di sicurezza, prestazioni e sviluppi futuri.

Indice

Abstract **i**

Sommario **ii**

1 Cenni storici su TCFS **6**

1.1 Il vecchio TCFS: origine e caratteristiche 6

1.1.1 Il contesto storico e le motivazioni del progetto 6

1.1.2 L'architettura e le funzionalità del vecchio TCFS 7

1.1.3 I limiti e le critiche al vecchio TCFS 7

1.2 Il nuovo TCFS: innovazione e sviluppo 7

1.2.1 Le ragioni e gli obiettivi della riscrittura del progetto 7

1.2.2 Le prospettive future del nuovo TCFS 9

1.2.3 Il contributo e il valore del nuovo TCFS 9

1.2.4 Le considerazioni personali e le lezioni apprese 10

2 Cos'è TCFS e come funziona **11**

2.1 Le differenze tra i filesystem non trasparenti e TCFS 11

2.1.1 Confronto con CODA 11

2.1.2 Confronto con AFS 12

2.1.3 Confronto con CFS 12

2.2 Caratteristiche di TCFS 12

2.3 Funzionamento e architettura di TCFS 13

2.3.1 L'architettura di TCFS e dove si trova nello stack 13

2.3.2 Il modulo userspace di TCFS 15

2.3.3 Il key manager di TCFS 16

2.3.4 Il server REST di TCFS 17

2.3.5 Logout 19

2.4 Svantaggi e vantaggi di TCFS 20

2.5 Sfide e problemi aperti di TCFS 20

3 AES, l'algoritmo di cifratura usato da TCFS 21

3.1 Definizione e caratteristiche di AES 21

3.1.1 AES CBC GCM e CTR 21

4 Read e Write in TCFS 27

4.1 Cenni generali sulla lettura e scrittura dei file in TCFS 27

4.1.1 La funzione do_crypt 28

4.1.2 La cifratura dei blocchi 28

4.1.3 La decifratura dei blocchi 29

4.2 L'implementazione della cifratura e della decifratura 29

4.3 Uso di AES CTR per cifrare e decifrare i file 35

4.3.1 La gerarchia e la generazione delle chiavi in TCFS 35

4.4 Uso di AES CBC per cifrare e decifrare i percorsi 36

4.4.1 Perchè AES CBC si scontra con la scrittura dei file in FUSE 37

4.4.2 Dettagli implementativi di encrypt_path_and_filename e decrypt_path_and_filename 37

4.4.3 Le funzioni ausiliarie encrypt_file_name_with_hex e decrypt_file_name_with_hex 40

5 Analisi delle prestazioni 45

5.1 Il Server per il Testing 45

5.2 La macchina client 46

5.3 Criteri e metodi di valutazione 46

5.3.1 Lettura e Scrittura 46

5.3.2 Il programma di testing 46

5.4 Valutazioni sulla velocità della rete e performance di TCFS 52

5.5 Risultati e discussione 53

5.5.1 TCFS utilizzando AES 256 CTR con NFS 53

5.5.2 BTFS su NFS senza criptazione 55

5.5.3 PTFS 58

5.6 Valutazione dei risultati 61

5.7 Possibili attacchi e contromisure 62

5.8 Valutazioni personali 62

6 Conclusioni 63

6.1 Limiti e prospettive future del lavoro 63

7 Glossario **65**

Elenco delle figure

2.1 Architettura di TCFS 14

3.1 Architettura AES CBC 22
3.2 Architettura AES GCM 23
3.3 Architettura AES CTR 25

Elenco delle tabelle

4.1 Descrizione dei blocchi nella WRITE 29
4.2 Descrizione dei blocchi nella READ 29
4.3 Descrizione dei blocchi nella READ-WRITE di AES CTR 35

Capitolo 1

Cenni storici su TCFS

Nella sua versione originale TCFS era molto diverso diverso da quello che è oggi, tutte le seguenti considerazioni in questo capitolo sono frutto dell'analisi di un unico documento, una delle ultime tracce rimaste della versione originale di TCFS. Il documento denominato: "The Design and Implementation of a Transparent Cryptographic Filesystem for UNIX" presentato nel 2001 allo USENIX è stato di forte ispirazione per il progetto. Sfortunatamente il codice originale è stato perso, questo ha comportato una ri-scrittura totale del FS.

1.1 Il vecchio TCFS: origine e caratteristiche

TCFS inizia a prendere forma ad UNISA, nel 1995, con lo scopo di rendere sicura la conservazione dei dati su macchine un pò diverse da quelle che abbiamo oggi, si intende dire che i computer erano basati su un'architettura in cui c'era una macchina principale e diversi terminali connessi. Questo era dovuto all'alto costo delle macchine dell'epoca, che rendeva impossibile avere un computer prestante e personale per ogni utente. Ad oggi diamo per scontato che il computer sia personale e che di solito l'utente connesso a una macchina sia singolo. Nella nuova versione di TCFS si sono tenuti in considerazione questi aspetti.

1.1.1 Il contesto storico e le motivazioni del progetto

La protezione dei dati memorizzati sul server era ed è tutt'ora una sfida importante, in quanto il server può essere soggetto a attacchi, guasti, o accessi non autorizzati. Inoltre, la comunicazione tra il client e il server può essere intercettata o alterata da terze parti malintenzionate. Per questi motivi si sentiva la necessità di sviluppare dei filesystem crittografici che potessero garantire la confidenzialità, l'integrità, e l'autenticazione dei dati, sia in transito che in riposo. I filesystem crittografici sono stati oggetto di molte ricerche negli ultimi decenni, a partire dalle prime proposte. Lungo questa linea di ricerca, la trasparenza (il fatto che il filesystem non sia locale dovrebbe essere nascosto alle applicazioni e agli utenti) e la sicurezza sono state le due questioni principali. Alcuni sistemi, come il Cluster File System di Compaq , garantiscono la trasparenza anche in presenza di guasti, ma assumono che la confidenzialità dei dati e l'autenticazione delle parti coinvolte sia ottenuta attraverso qualche altro sistema. Il File System di Andrew e CODA invece forniscono dei meccanismi per autenticare i server su

linee pubbliche e per assicurare che la comunicazione client-server non possa essere intercettata. Le serie questioni di gestione delle chiavi che sorgono in tali file system sono affrontate dal Self-certifying File System. Facciamo notare che sia AFS che CODA assumono che il server sia affidabile e memorizzano i dati in chiaro sulla macchina server. TCFS invece memorizza i file in forma criptata, negando al server l'accesso ai dati in chiaro.

1.1.2 L'architettura e le funzionalità del vecchio TCFS

Date le tecnologie dell'epoca TCFS era strutturado in maniera completamente diversa rispetto ad oggi. Esso era costituito da un modulo kernel scritto "ad hoc", che si occupava di memorizzare e mantenere in maniera sicura le chiavi degli utenti all'interno del kernel; utilizzava systemcall, anch'esse scritte appositamente per TCFS, per richiedere e salvare le chiavi degli utenti. Inoltre il FS era capace di comunicare solo con NFS, difatti era una versione modificata proprio di NFS.

1.1.3 I limiti e le critiche al vecchio TCFS

Nonostante TCFS fosse un passo avanti per l'epoca ad oggi appaiono palesi alcune limitazioni date dall'avanzare dei tempi. Con questo si intende che TCFS era limitato alla comunicazione tramite NFS, richiedeva la ri-compilazione del kernel per aggiungere la gestione delle chiavi e richiedeva che le funzioni crittografiche fossero scritte da zero ed internamente al kernel, tutte queste limitazioni sono state superate dalla versione attuale di TCFS. Per quanto riguarda le critiche alla versione originale di TCFS abbiamo che "*TCFS presenta diversi punti deboli che lo rendono meno utile per la distribuzione. In primo luogo, il ricorso alle password di accesso come chiavi utente non è sicuro. Inoltre, la memorizzazione delle chiavi di crittografia su disco in un database di chiavi riduce ulteriormente la sicurezza. Infine, TCFS è disponibile solo su sistemi con kernel Linux 2.2.17 o precedenti, il che ne limita la disponibilità.*"[8]

1.2 Il nuovo TCFS: innovazione e sviluppo

La nuova versione di TCFS basa la sua implementazione su FUSE, che ha permesso di portare il codice in userspace, utilizzare librerie crittografiche (nel nostro caso fornite da openssl), e di svincolarci da NFS. Di fatti la nuova versione può funzionare insieme a qualsiasi tipo di FS a patto che supporti gli extended attributes. Nella pratica possiamo immaginarlo come un overlay posto al di sopra della cartella sulla quale viene montato TCFS. Lo sviluppo è stato al dir poco travagliato e ha richiesto la costante e continua ricerca delle nuove tecnologie e di come queste potessero essere adattate all'idea proposta. La nuova gestione del keyring di Linux ha permesso di eliminare la necessità di un modulo kernel per la gestione delle chiavi utente, FUSE ha permesso di portare il codice in userspace, MariaDB e GO hanno permesso un implementazzione del threshold sharing semplificata. Il progetto è stato totalmente reinventato.

1.2.1 Le ragioni e gli obiettivi della riscrittura del progetto

Come specificato precedentemente il codice originale di TCFS è stato perso, per questo motivo si è dovuto riscrivere interamente il FS. Chiaramente dato l'avanzare dei tempi si è

ritenuto necessario indagare le nuove tecnologie proposte da quest'epoca, ma si è anche voluto rispettare quanto più possibile il disegno originale, tenendo a mente i seguenti obbiettivi:

- **Portabilità**: Tutte le tecnologie utilizzate da TCFS sono disponibili per OS UNIX-based, cioè Linux, MacOS, NetBSD, OpenBSD, FreeBSD e similari, ma anche per WINDOWS e Solaris, ed è stato scritto in linguaggi portabili, C e GO, rendendo il porting verso nuove architetture plausibile.

- **Semplicità**: TCFS vuole essere semplice nella sua architettura, semplice nel debug e semplice da utilizzare per l'utente.

- **Trasparenza**: La trasparenza in un FS è un concetto fondamentale che richiede una comprensione approfondita. Quando parliamo di trasparenza in un FS, ci riferiamo alla capacità del sistema di nascondere la complessità delle sue operazioni interne all'utente e in questo caso anche al server remoto, rendendo l'interazione con il sistema il più semplice e intuitiva possibile. La creazione di un FS che sia totalmente trasparente all'utente non è un compito semplice per diversi motivi: Le operazioni che un FS esegue in background sono estremamente complesse. Queste includono la gestione dello spazio su disco, l'organizzazione dei file, la gestione dei permessi di accesso, e numerose altre. Nascondere queste operazioni all'utente richiede un design accurato e un'implementazione meticolosa. Quando un FS interagisce con un server remoto, ci sono molte variabili da considerare. Queste includono la latenza della rete, la gestione degli errori di rete, la sincronizzazione dei file tra il client e il server, ecc. Decidere di non applicare modifiche a un server remoto può sembrare una soluzione facile, ma in realtà, raggiungere una trasparenza totale sia verso il server che verso l'utente richiede di tenere conto di queste variabili. Un altro aspetto importante della trasparenza è la gestione degli errori. Un FS deve essere in grado di gestire gli errori in modo tale da minimizzare l'impatto sull'utente. Questo può includere il recupero da errori di sistema, la gestione di errori di rete, la prevenzione della perdita di dati, ecc. Infine, un FS deve essere in grado di fornire un alto livello di performance pur mantenendo la trasparenza. Questo significa che deve essere in grado di gestire grandi quantità di dati, eseguire operazioni in modo efficiente e rispondere rapidamente alle richieste dell'utente. In conclusione, la trasparenza in un FS è un obiettivo difficile da raggiungere a causa della complessità delle operazioni coinvolte e delle sfide associate all'interazione con i server remoti. Tuttavia, con un design accurato e un'implementazione meticolosa, è possibile creare un FS che fornisca un alto livello di trasparenza all'utente.

- **Sostenibilità**: La sostenibilità è un aspetto cruciale nella scrittura di un progetto software. Essa implica la capacità del progetto di resistere nel tempo, adattandosi alle nuove tecnologie e alle esigenze mutevoli degli utenti. La sostenibilità di un progetto software può essere influenzata da vari fattori, tra cui l'uso accurato delle tecnologie disponibili, la manutenibilità del codice e la capacità di adattarsi alle nuove versioni delle dipendenze. Nel caso specifico di TCFS, la versione originale è stata sviluppata come modulo del kernel. Questo significa che ogni volta che viene rilasciata una nuova versione del kernel, è necessario rivedere e potenzialmente aggiornare l'implementazione di TCFS.

Questo può rendere la manutenzione del progetto difficile e laboriosa. Tuttavia, la nuova versione di TCFS supera queste limitazioni. Invece di richiedere che il team di TCFS mantenga il modulo del kernel, questa responsabilità viene delegata agli sviluppatori del kernel stesso. Questo significa che, presumibilmente, il modulo del kernel sarà supportato per tutta la sua vita, indipendentemente dalle modifiche al kernel. Questo approccio ha diversi vantaggi. Prima di tutto, riduce il carico di lavoro del team di TCFS, permettendo loro di concentrarsi su altre aree del progetto, non aumentando il carico sugli sviluppatori del kernel. In conclusione, la sostenibilità di un progetto software è un obiettivo importante che può essere raggiunto attraverso un uso accurato delle tecnologie disponibili e un'attenta pianificazione della manutenzione e dell'evoluzione del progetto. Nel caso di TCFS, la delega della manutenzione del modulo del kernel agli sviluppatori del kernel stesso è un esempio di come questo obiettivo possa essere raggiunto a costo zero.

1.2.2 Le prospettive future del nuovo TCFS

Al momento di redazione di questa tesi l'implementazione di TCFS risulta incompleta, di fatti non supporta nessun meccanismo di key-recovery e l'implementazione del threshold sharing è parziale, inoltre non è stato svilupparo nessun programma che aiuti l'utente a montare il FS; un altro passo importante è l'utilizzo di funzioni di lettura e scrittura crittografiche che supportino il parallelismo. Ci si prospetta l'implementazione di queste funzionalità nelle versione successive di TCFS.

1.2.3 Il contributo e il valore del nuovo TCFS

L'implementazione del nuovo TCFS si colloca in un contesto storico-sociale in cui la salvaguardia dei dati personali è emersa come un problema di rilevanza critica. Questa problematica è particolarmente accentuata nell'attuale era digitale, caratterizzata dalla presenza di conglomerati tecnologici che acquisiscono e analizzano dati personali per scopi di lucro. In questo scenario, la privacy appare come un concetto che rischia di essere trascurato. Come emerge da diverse fonti [10] [5] [4] [1], le Big Tech sono in grado, attraverso l'utilizzo di strumenti e risorse, capaci di ricavare valore dai dati in loro possesso, di offrire i nostri dati agli advertisers. Tuttavia, questo comporta anche dei rischi e delle sfide, sia per gli utenti che per le autorità, in termini di: diritti e libertà fondamentali della persona; sicurezza e integrità dei dati; concorrenza e mercato; etica e responsabilità. In questo scenario, il nuovo TCFS offre agli utenti la possibilità di avere il controllo sui propri dati, e di decidere chi e come può accedervi, senza dover rinunciare alla comodità e alla trasparenza di un filesystem tradizionale. In questo modo, il nuovo TCFS contribuisce a rafforzare il principio di protezione delle persone fisiche con riguardo al trattamento dei dati di carattere personale, sancito dal GDPR, e a promuovere una maggiore consapevolezza e gestione dei rischi e della sicurezza legati ai dati.

Uno degli aspetti più importanti e distintivi del nuovo TCFS è la sua capacità di criptare i dati prima di inviarli su un server remoto, tramite protocolli di rete. In questo modo, TCFS garantisce che i dati siano protetti sia in transito che in riposo, cioè sia durante la comunicazione tra il client e il server, sia quando sono memorizzati sul server. Questa funzionalità rende

TCFS adatto per l'uso di filesystem di rete, che offrono dei vantaggi, come la condivisione, la scalabilità, e la disponibilità dei dati, ma presentano anche dei rischi, come la perdita, il furto, o la corruzione dei dati, dovuti a possibili attacchi o malfunzionamenti della rete. Con il nuovo TCFS, gli utenti possono usufruire dei servizi offerti dalle grandi aziende tecnologiche, senza dover esporre i propri dati in chiaro, e senza dover affidare la loro confidenzialità a terze parti. In questo modo, il nuovo TCFS tutela la privacy degli utenti, e li rende più indipendenti e sicuri.

1.2.4 Le considerazioni personali e le lezioni apprese

Nell'ambito dell'attuale ecosistema tecnologico, l'insorgere di progetti come TCFS appare non solo appropriato, ma imprescindibile. L'appeal di servizi di hosting gratuiti è indiscutibilmente rilevante, tuttavia, è essenziale riconoscere che in tali contesti, l'utente si trasforma inevitabilmente nel prodotto. È altrettanto rilevante sottolineare che, nonostante l'esistenza di servizi di hosting a pagamento, la riservatezza dei dati non è assicurata in quasi nessuna circostanza. Pertanto, l'integrazione di TCFS in questo scenario si rivela di fondamentale importanza. Grazie alla sua eccezionale facilità d'uso, TCFS può essere adoperato anche da utenti con competenze limitate, offrendo una soluzione semplice ma efficace alle sfide inerenti alla privacy che caratterizzano l'era digitale contemporanea.

L'implementazione di questo progetto, come precedentemente accennato, ha comportato notevoli sfide. Questo è dovuto alla vastità del progetto e alla ricerca delle tecnologie appropriate, che ha richiesto un impegno temporale considerevole. Inoltre, l'implementazione degli algoritmi crittografici di OpenSSL si è rivelata un compito arduo. L'acquisizione della padronanza di tutte le tecnologie coinvolte è stata un'esperienza estremamente arricchente. Concludo questo progetto con una consapevolezza più profonda della complessità di AES, delle sfide relative alla privacy, del kernel Linux e dei linguaggi di programmazione C, Go e SQL.

Capitolo 2

Cos'è TCFS e come funziona

TCFS (Transparent cryptografic file system) è un FS crittografico con lo scopo di rendere totalmente trasparente all'utente le operazioni crittografiche sottostanti. Un filesystem crittografico permette all'utente di proteggere i dati rendondoli illegibili ad utenti non autorizzati. Molti FS crittografici richiedono all'utente di eseguire comandi particolari per gestire i file inclusi al loro interno, TCFS evita queste scomodità ed evita di dover ricordare una password per accedere ai contenuti, evitando di compromettere la sicurezza. TCFS sfrutta le funzionalità di FUSE per interporsi nella comunicazione tra userspace e kernel in modo da cifrare e decifrare i contenuti dei file e dei nomi dei file. Questo FS ha lo scopo principale di criptare i dati prima di inviarli su un server remoto, tramite protocolli di rete come NFS o webdav e simili, a patto che supportino gli attributi estesi. In questo modo, TCFS garantisce che i dati siano protetti sia in transito che in riposo, cioè sia durante la comunicazione tra il client e il server, sia quando sono memorizzati sul server. L'architettura di TCFS presuppone che il client sia fidato, ma chiaramente non si fida del server. Un aspetto distintivo, sebbene non primario dell'implementazione di TCFS, basata su FUSE, è la capacità tecnica di cifrare anche file in directory locali. Questo rappresenta un valore aggiunto significativo, poiché estende la protezione dei dati oltre il contesto di rete, garantendo la sicurezza dei dati anche a livello locale. In conclusione, l'approccio adottato da TCFS offre un equilibrio ottimale tra facilità d'uso, versatilità e sicurezza dei dati, rendendolo una soluzione ideale per affrontare le sfide della privacy odierna.

2.1 Le differenze tra i filesystem non trasparenti e TCFS

I filesystem crittografici non trasparenti richiedono che l'utente esegua comandi specifici per gestire i file all'interno del sistema, mentre TCFS (Transparent Cryptographic File System) rende queste operazioni crittografiche completamente trasparenti all'utente. Questa trasparenza offre un equilibrio ottimale tra facilità d'uso, versatilità e sicurezza dei dati.

2.1.1 Confronto con CODA

CODA è un sistema di file distribuito sviluppato presso l'Università di Carnegie Mellon che mira a fornire un sistema di file altamente disponibile e scalabile. A differenza di TCFS,

CODA non è progettato specificamente per fornire funzionalità crittografiche, tuttavia, CODA supporta la crittografia per la sicurezza dei dati durante la trasmissione. A differenza di CODA, TCFS assicura che i dati sulla rete e a riposo siano crittografati. Mentre TCFS si concentra sulla trasparenza delle operazioni crittografiche per l'utente, CODA si concentra sulla fornitura di un sistema di file distribuito robusto e resiliente. CODA utilizza tecniche come la replica dei server e la cache del client per fornire accesso ai file anche quando la rete è instabile o non disponibile.

2.1.2 Confronto con AFS

L'Andrew File System (AFS) è un filesystem distribuito che utilizza un insieme di server fidati per presentare uno spazio dei nomi dei file omogeneo e indipendente dalla posizione a tutte le workstation client. Tuttavia, a differenza di TCFS, AFS non fornisce funzionalità crittografiche trasparenti [8].

2.1.3 Confronto con CFS

Il Cryptographic File System (CFS) è un altro sistema di file crittografico che fornisce la crittografia a livello di file. A differenza di TCFS, CFS non è progettato per essere completamente trasparente per l'utente. Invece, CFS richiede che l'utente monti esplicitamente il filesystem crittografico e fornisca una chiave di crittografia. Mentre TCFS si concentra sulla trasparenza delle operazioni crittografiche per l'utente, CFS si concentra sulla sicurezza dei dati. CFS utilizza una chiave di crittografia per crittografare i dati, il che significa che solo gli utenti che conoscono la chiave possono accedere ai dati. In conclusione, sia TCFS che CFS offrono funzionalità uniche e preziose. TCFS si distingue per la sua trasparenza nelle operazioni crittografiche, rendendo facile per gli utenti proteggere i loro dati senza dover gestire direttamente la crittografia. Ambedue offrono un alto livello di sicurezza dei dati, con l'uso di chiavi di crittografia per proteggere l'accesso ai dati, ma TCFS lo fa in maniera trasparente. [9].

2.2 Caratteristiche di TCFS

TCFS pone come carattiristiche principali:

- **Trasparenza**: TCFS rende la cifratura e la de-cifratura dei file invisibili all'utente, che può usare il filesystem ignaro della complessità sottostante, senza dover inserire password o usare comandi specifici.
- **Sicurezza**: TCFS usa l'algoritmo AES GCM per criptare i file, considerato uno dei più sicuri e diffusi al mondo, usando delle chiavi generate casualmente, in modo da ridurre il rischio di perdita o furto delle chiavi. Inoltre questo algoritmo è spesso utilizzato date le sue ottime performance.
- **Flessibilità**: TCFS si basa su FUSE, una tecnologia che permette di creare e gestire dei filesystem a livello utente, senza dover modificare e ricompilare il kernel del sistema

operativo. Questo rende TCFS compatibile con diversi supporti fisici e portabile verso diversi OS Inoltre e permette di personalizzare il filesystem sottostante (riferirsi alla figura 2.1) in base alle proprie esigenze.

2.3 Funzionamento e architettura di TCFS

TCFS è diviso in varie parti, un modulo userspace che si interfaccia con FUSE, un server REST per il threshold sharing e un programma helper che aiuta l'utente ad avviare TCFS. Ogni parte in TCFS è di fatto indipendente dalle altre, di seguito viene illustrata in maggiore dettaglio l'architettura del FS.

2.3.1 L'architettura di TCFS e dove si trova nello stack

Il modulo TCFS agisce come un intermediario tra l'utente e il modulo kernel di FUSE. Questo posizionamento strategico consente a TCFS di intercettare e gestire le invocazioni dei comandi che operano sui file. Una volta che TCFS ha elaborato le operazioni, le inoltra al kernel, che a sua volta esegue l'operazione corrispondente sul file.

Per illustrare meglio questo processo, consideriamo due esempi comuni: la lettura e la scrittura di un file.

- **Lettura di un file**: Quando un utente tenta di leggere un file, l'invocazione del comando di lettura viene prima intercettata dal modulo kernel di FUSE (grazie a VFS), che inoltra a TCFS la richiesta di lettura. TCFS quindi decifra il contenuto del file e popola un buffer, preallocato dal modulo kernel, con i dati decifrati. Successivamente vengono restituiti i dati decifrati all'utente.

- **Scrittura di un file**: Analogamente, quando un utente scrive su un file, l'invocazione del comando di scrittura viene intercettata da TCFS, che cifra i dati e inoltra i dati cifrati al modulo kernel di FUSE, che scrive i dati cifrati sul file.

In entrambi i casi, TCFS garantisce che i dati siano sempre cifrati quando sono memorizzati sul file, fornendo così una protezione crittografica dei dati. Questo meccanismo di interposizione tra l'utente e il kernel consente a TCFS di fornire un livello aggiuntivo di sicurezza e privacy, proteggendo i dati dell'utente da accessi non autorizzati.

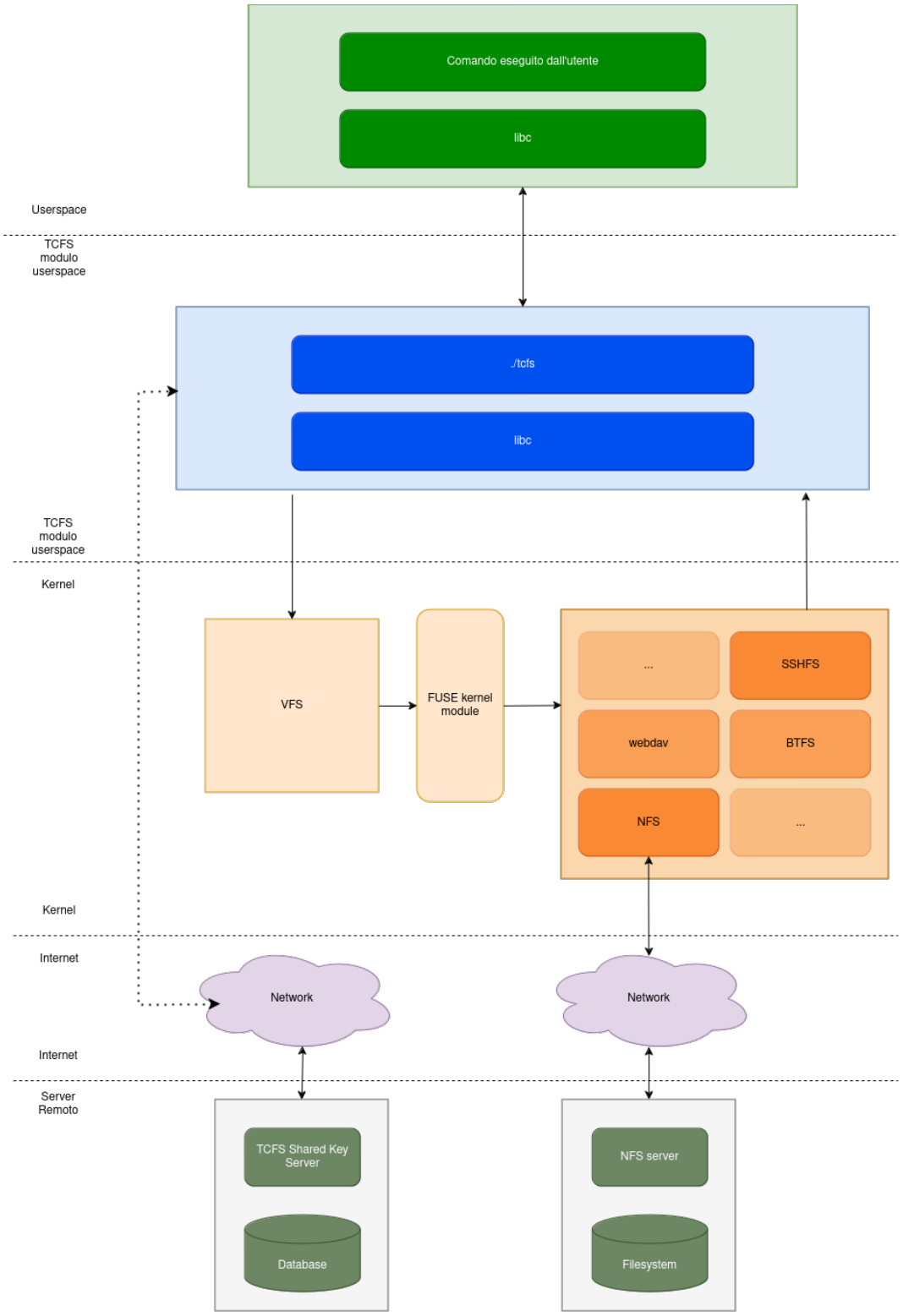


Figura 2.1: Architettura di TCFS

Chiarimenti su VFS e FUSE

Il Virtual File System (VFS) è un'interfaccia astratta a livello del kernel del sistema operativo che fornisce un meccanismo per l'interazione uniforme con vari filesystem. Il VFS è responsabile della gestione e coordinamento delle richieste di I/O tra il kernel del sistema

operativo e i vari filesystem sottostanti. Esso fornisce un insieme comune di API per le operazioni sui file, indipendentemente dal tipo di filesystem sottostante. Questo livello di astrazione consente al sistema operativo di supportare una vasta gamma di filesystem, come ext4, NTFS, FAT32, ecc., senza dover implementare le specifiche di ciascuno di essi nel kernel. FUSE (Filesystem in Userspace) è un modulo del kernel che fornisce un'interfaccia per la creazione di filesystem personalizzati in spazio utente. FUSE estende le funzionalità del VFS permettendo agli utenti di implementare un filesystem come un'applicazione a livello di utente, piuttosto che come un modulo del kernel. Questo offre una maggiore flessibilità e sicurezza, poiché gli errori nel codice del filesystem non possono causare crash del sistema. FUSE si interfaccia con il VFS tramite un'API specifica. Quando un processo effettua una richiesta di I/O su un file o una directory all'interno di un filesystem FUSE, il VFS inoltra la richiesta al modulo FUSE. FUSE a sua volta traduce la richiesta in un formato comprensibile per l'applicazione del filesystem FUSE e la inoltra all'applicazione. L'applicazione del filesystem FUSE, che è un'applicazione standalone che utilizza la libreria libfuse, implementa la logica specifica del filesystem. Questa applicazione gestisce le richieste provenienti da FUSE, elabora i dati necessari e invia una risposta a FUSE. Ad esempio, se viene richiesta la lettura di un file, l'applicazione del filesystem FUSE leggerà il file dal disco, elaborerà i dati se necessario, e invierà i dati letti a FUSE come risposta. FUSE quindi restituisce la risposta al VFS, che a sua volta la restituisce al processo che ha effettuato la richiesta originale. In questo modo, FUSE agisce come un ponte tra il VFS a livello di kernel e il filesystem implementato in spazio utente. In sintesi, il VFS, FUSE e l'applicazione del filesystem FUSE lavorano insieme per fornire un'interfaccia uniforme per la gestione dei filesystem, consentendo al contempo la creazione di filesystem personalizzati in spazio utente. Questo approccio oltre a fornire una grande flessibilità, offre anche un modello di progettazione particolarmente utile, flessibile e semplificato per lo sviluppo di nuovi filesystem, la prototipazione di funzionalità del filesystem e la creazione di filesystem per scopi specifici.

2.3.2 Il modulo userspace di TCFS

FUSE mette a disposizione una struttura dati che mappa le funzioni specificate da TCFS con le API di FUSE. Nel caso di TCFS le operazioni supportate sono:

Listing 2.1: Mappa FUSE API alle funzioni di TCFS

```
1 static struct fuse_operations tcfs_oper = {
2   .opendir = tcfs_opendir,
3   .getattr = tcfs_getattr,
4   .access = tcfs_access,
5   .readlink = tcfs_readlink,
6   .readdir = tcfs_readdir,
7   .mknod = tcfs_mknod,
8   .mkdir = tcfs_mkdir,
9   .symlink = tcfs_symlink,
10  .unlink = tcfs_unlink,
11  .rmdir = tcfs_rmdir,
```

```
12    .rename = tcfs_rename,
13    .link = tcfs_link,
14    .chmod = tcfs_chmod,
15    .chown = tcfs_chown,
16    .truncate = tcfs_truncate,
17    .utimens = tcfs_utimens,
18    .open = tcfs_open,
19    .read = tcfs_read,
20    .write = tcfs_write,
21    .statfs = tcfs_statfs,
22    .create = tcfs_create,
23    .release = tcfs_release,
24    .fsync = tcfs_fsync,
25    .setxattr = tcfs_setxattr,
26    .getxattr = tcfs_getxattr,
27    .listxattr = tcfs_listxattr,
28    .removexattr = tcfs_removexattr,
29 };
```

Nel contesto di FUSE, quando si parla di funzioni, ci si riferisce alle funzioni di callback che l'utente deve implementare per definire il comportamento del FS. Queste funzioni vengono poi chiamate da FUSE quando si verificano determinate operazioni sul FS, come la lettura di un file o la creazione di una directory. Ogni funzione di callback ha un insieme specifico di parametri che deve accettare. Questi parametri sono definiti da FUSE e possono includere informazioni come il percorso di un file o la dimensione di un buffer di lettura. Il numero e il tipo di questi parametri sono determinati dalla specifica operazione sul FS che la funzione deve gestire. Inoltre, FUSE può pre-allocare o pre-riempire alcuni di questi parametri con i dati necessari prima di chiamare la funzione di callback. Ad esempio, se la funzione di callback è responsabile della lettura di un file, FUSE potrebbe pre-allocare un buffer di lettura e passarlo come parametro alla funzione. In sintesi, FUSE specifica quali parametri una funzione deve accettare e può anche pre-allocare o pre-riempire questi parametri con i dati necessari prima di chiamare la funzione. Questo permette all'utente di concentrarsi sulla logica specifica del FS, senza doversi preoccupare della gestione dei dettagli a basso livello.

2.3.3 Il key manager di TCFS

TCFS custodisce le chiavi dell'utente all'interno del kernel sfruttando il set di utilities Keyutils, incluso in Linux. Keyutils è un insieme di utility per la gestione e conservazione delle chiavi nel kernel, esso permette di mantenere una chiave all'interno del keyring dell'OS; questa chiave è identificata da in key_id, che viene specificato nel file di configurazione di TCFS. Il fetch della chiave viene eseguito dall'apposita systemcall keyctl_read e la chiave viene concessa se e solo se l'ID della chiave è valido e l'utente ha i permessi necessari per richiederla. In generale, l'utente che ha inserito la chiave nel keyring ha i permessi per accedervi, ma può anche concederli o revocarli ad altri utenti, usando il comando keyctl setperm. Chiaramente è necessario ricordare che per definizione dell'architettura di TCFS l'utente è sempre fidato,

dunque come abbia ottenuto la chiave non impatta sulla sicurezza del progetto. Nello specifico è stata sviluppata una libreria denominata key_utils che espone tre funzioni:

- **char *get_key ()**:che ottiene la chiave dal kernel e restituisce un buffer contenente la chiave stessa. Il buffer viene allocato dal kernel e condiviso con TCFS.
- **void free_key (const char *key)**: Che prende in input il puntatore alla chiave, sovrascrive l'area di memoria con "0" e libera lo spazio della chiave.
- **void set_key_id(const char *value)**: Che permette di settare l'id della chiave per la sessione attuale di TCFS.

2.3.4 Il server REST di TCFS

Durante il corso del progetto è stato sviluppato un REST in Golang atto a gestire il threshold sharing attraverso l'algoritmo di Shamir. Questo consente di garantire la sicurezza e la riservatezza nell'accesso ai file soggetti al threshold sharing. Si ricorda che questo server non è necessario per il funzionamento di TCFS che ne è di fatti indipendente. Quando un file viene condiviso utilizzando il threshold sharing la chiave del file viene suddivisa in multiple parti, e ciascuna parte è associata a un share univoco. Il file completo può essere ricostruito solo quando un numero specificato di share viene combinato insieme, rispettando il threshold definito. Non è stato implementato un modo per riottenere la chiave del file.

Cos'è l'algoritmo di Shamir e come funziona

Il threshold sharing, ottenuto tramite l'algoritmo di Shamir è un concetto fondamentale nell'ambito della crittografia e della sicurezza informatica. Esso consente di suddividere un segreto in multiple parti, note come "share", in modo che il segreto possa essere ricostruito solo quando un numero minimo di share viene combinato insieme. Questo numero minimo è definito come "threshold". Questa metodologia fornisce un elevato livello di sicurezza, in quanto anche se alcuni share vengono compromessi o ottenuti da parte di terzi non autorizzati, il segreto non può essere ricostruito senza il numero minimo di share richiesto. Pertanto, l'implementazione del threshold sharing tramite l'algoritmo di Shamir nel server REST di TCFS contribuisce in modo significativo alla protezione dei dati e alla sicurezza delle operazioni di accesso ai file. L'algoritmo di Shamir, fondamentale nell'ambito della crittografia e della sicurezza informatica, si basa sulla matematica dell'interpolazione polinomiale. Per un segreto S l'algoritmo genera un polinomio di grado $(t-1)$, dove t è la soglia. Il termine costante del polinomio è il segreto S, mentre gli altri coefficienti del polinomio sono scelti casualmente. Ogni share è quindi un punto $(x, f(x))$ sul polinomio, dove x è un valore unico per ogni share e $f(x)$ è il valore del polinomio calcolato in x. Il segreto S può essere ricostruito combinando t share usando l'interpolazione polinomiale di Lagrange.

Più precisamente, l'algoritmo di Shamir procede come segue:

1. Scegliamo un numero primo grande p tale che $p > n$ e $p > S$, dove n è il numero totale di share che vogliamo creare.

2. Creiamo un polinomio di grado $(t-1)$ nel seguente modo:

$$f(x) = S + a_1 x + a_2 x^2 + \ldots + a_{t-1} x^{t-1} \mod p$$

dove S è il termine costante e $a_1, a_2, \ldots, a_{t-1}$ sono coefficienti scelti casualmente.

3. Ogni share è un punto $(x, f(x))$ sul polinomio, dove x è un valore unico per ogni share e $f(x)$ è il valore del polinomio calcolato in x.

4. Il segreto S può essere ricostruito combinando t share usando l'interpolazione polinomiale di Lagrange. La formula per l'interpolazione di Lagrange è la seguente:

$$L(x) = \sum_{i=1}^{t} y_i \prod_{1 \le j \le t, j \neq i} \frac{x - x_j}{x_i - x_j} \mod p$$

dove (x_i, y_i) sono le coordinate dei share. Calcolando $L(0)$ otteniamo il segreto S.

La sicurezza dell'algoritmo di Shamir si basa sulla difficoltà di risolvere il problema dell'interpolazione polinomiale in presenza di rumore, che è noto come problema di interpolazione di Reed-Solomon. Questo problema è noto per essere NP-hard, il che significa che non esiste un algoritmo efficiente per risolverlo se P $\neq$ NP.

Un problema NP-hard è un problema di decisione per il quale non esiste un algoritmo deterministico che lo risolve in tempo polinomiale. In altre parole, un problema è NP-hard se ogni problema in NP può essere ridotto ad esso in tempo polinomiale. La classe di problemi NP-hard rappresenta quindi i problemi di decisione più difficili in NP. Nonostante la loro difficoltà, i problemi NP-hard sono importanti in quanto forniscono un modo per classificare i problemi computazionali in base alla loro difficoltà intrinseca.

Registrazione

Il server si aspetta che ogni utente con il quale si voglia condividere la chiave sia stato registrato. La registrazione avviete tramita una chiamata API a "/register", si specificano il nome utente, la password e la pubkey. Le informazioni vengono deserializzate dal server e salvate nel database, chiaramente non viene salvata la password in chiaro ma un hash della password.

Login

Il login all'interno del server è stato implementato ma attualmente non ha nessuna utilità, servirà a riottenere il proprio segreto di Shamir quando sarà terminata l'implementazione del threshold sharing. L'implementazione è molto semplice, l'utente specifica il proprio username e la propria password, quindi il server cerca di eseguire un match con i dati salvati sul database, se questo ha successo allora l'utente viene considerato "loggato" e aggiunto alla tabella corrispondente.

2.3.5 Logout

Il logout avviene in maniera similare al login, viene specificato il nome utente e la password e se sono corretti l'uente viene considerato non più "loggato" e rimosso dalla tabella corrispondente

Come si condivide un file

La procedura per richiedere la condivisione di un file si svolge attraverso una chiamata API a "/createSharedFile". Durante questa chiamata, si specificano due parametri chiave: il numero di share che si desidera generare e il threshold necessario per ricostruire la chiave principale. In risposta a questa richiesta, il server fornisce un ID univoco associato al file, che deve essere conservato all'interno degli attributi estesi del file. Quando il server riceve questa richiesta, esegue due operazioni fondamentali. Prima di tutto deserializza la richiesta, convertendo i dati in un formato utilizzabile. Successivamente, popola due strutture dati con le informazioni estratte dalla richiesta. Questo processo dettagliato assicura che tutte le informazioni necessarie siano accuratamente raccolte e pronte per l'uso nelle fasi successive del processo di condivisione del file. Segue una breve descrizione del codice:

Listing 2.2: Strutture dati ottenute

```
type User struct {
        Username string `json:"username"`
    }

    type Request struct {
        Users []User `json:"users"`
        K     int    `json:"k"`
    }
```

Ottenute queste informazioni si ricostruiscono gli utenti coinvolti nello share grazie al seguente frammento di codice:

Listing 2.3: Ottenimento informazioni utenti

```
var users []TCFSTypes.TCFSUser
    for _, user := range req.Users {
        tmpUser := TCFSTypes.TCFSUser{Username: user.Username}
        err := DB.LoadUserInfoByName(&tmpUser)
        if err != nil {
            http.Error(w, err.Error(), http.StatusBadRequest)
            fmt.Printf("Could not load user %v info %v\n", user,
                err)
            return
        }
```

```
10          users = append(users, tmpUser)
11      }
```

Grazie al quale controlliamo se tutti gli utenti che sono coinvolti nello share esistano all'interno del database del server. A questo punto il server procede a generare una chiave chiave, suddividerla tramite Shamir e salvare ogni pezzo della chiave all'interno del suo database, criptandola con la chiave pubblica associata a ogni utente.

2.4 Svantaggi e vantaggi di TCFS

Ovviamente TCFS aggiunge un livello di complessità non banale allo share remoto. Gli algoritmi per criptare i file sono molto sicuri, si difendono egragiamente da diversi attacchi, ma questa complessità aggiunge un overhead alla scrittura e lettura dei file, rendendo tutte le operazioni più lente. Questo grande svantaggio viene però compensato dalla sicurezza che tutti i dati presenti sullo share remoto e tutti i dati in transito nella rete siano criptati e indecifrabili per un attaccante.

2.5 Sfide e problemi aperti di TCFS

Attualmente TCFS non ha grandi problematiche aperte, tuttavia ci sono numerose sfide minori da affrontare. Queste sono:

- **Gestione degli IV**:Attualmente, TCFS non implementa un uso appropriato degli IV nella cifratura dei file. Inoltre, i nomi dei file non utilizzano un IV nel loro processo di cifratura. È importante sottolineare che una corretta gestione degli IV è fondamentale per garantire la sicurezza del sistema. Questo perché l'uso di IV in modo appropriato assicura che la cifratura non sia deterministica, un aspetto cruciale per prevenire attacchi crittanalitici. Pertanto, è essenziale che TCFS rivisiti e migliori la sua gestione degli IV per rafforzare la sicurezza complessiva del sistema.

- **Implementazione del threshold sharing**: Come accennato precedentemente il threshold sharing non è stato implementato completamente. Allo stato attuale non è stata presa nessuna decisione su come ricostruire le chiavi.

- **Parallelismo**: Nessuno degli algoritmi di cifratura di TCFS supporta il parallelismo allo stato attuale del progetto. Una corretta implementazione di questi è di fondamentale importanza per velocizzare le operazioni di IO dal filesystem.

Capitolo 3

AES, l'algoritmo di cifratura usato da TCFS

Nel corso dell'implementazione di TCFS sono stati implementati diversi algoritmi di AES per la cifratura dei file, sono stati valutati CBC, GCM, CTR, e parallelamente, l'AES 256 CBC per la cifratura dei nomi delle cartelle e i nomi dei file. Questo capitolo è dedicato all'analisi delle decisioni progettuali che hanno guidato la selezione di questi algoritmi crittografici e alla discussione dettagliata della loro implementazione all'interno di TCFS. L'obiettivo è di fornire una comprensione approfondita delle motivazioni dietro queste scelte e di come esse contribuiscono a migliorare la sicurezza e l'efficienza del nostro sistema.

3.1 Definizione e caratteristiche di AES

L' **Advanced Encryption Standard (AES)** è un algoritmo di cifratura a blocchi utilizzato per crittografare dati sensibili. Le sue principali caratteristiche includono:

- **Robustezza**: AES è noto per la sua resistenza agli attacchi crittografici.
- **Efficienza**: L'algoritmo AES è progettato per essere veloce ed efficiente.
- **Flessibilità**: Supporta chiavi di diversi bit (128, 192, 256 bit) offrendo flessibilità nella sicurezza.
- **Standardizzazione**: Essendo uno standard internazionale, AES è ampiamente supportato.

3.1.1 AES CBC GCM e CTR

In questa sezione analizzeremo le differenze tra AES CBC, GCM e CTR, i loro vantaggi e svantaggi, e le loro applicazioni per giustificare la scelta implementativa di diffirenziare l'algoritmo di crifratura per i nomi delle cartelle e dei file.

AES CBC

AES CBC è un modo di operazione che usa una catena di blocchi per cifrare i dati. Ogni blocco di testo in chiaro viene sottoposto a un'operazione di XOR (OR esclusivo) con il blocco di testo cifrato precedente, o con un vettore di inizializzazione (IV) nel caso del primo blocco, prima di essere cifrato con la chiave AES. Questo modo garantisce che due blocchi uguali di testo in chiaro non producano lo stesso blocco di testo cifrato, aumentando la sicurezza. Tuttavia, AES CBC ha anche alcuni svantaggi:

- AES CBC non è parallelizzabile, cioè non è possibile cifrare o decifrare più blocchi contemporaneamente, perché ogni blocco dipende dal precedente. Questo riduce le prestazioni, soprattutto con dati di grandi dimensioni.
- AES CBC non fornisce autenticazione, cioè non è possibile verificare se il testo cifrato è stato alterato o manipolato da un attaccante. Si potrebbe usare AES CBC in combinazione con un MAC (Message Authentication Code), che è un codice che garantisce l'integrità dei dati. Questo richiede però una chiave aggiuntiva e un'operazione in più, aumentando la complessità e il costo computazionale.

AES CBC è adatto per cifrare file locali che non richiedono accesso casuale, cioè che vengono letti o scritti sequenzialmente.

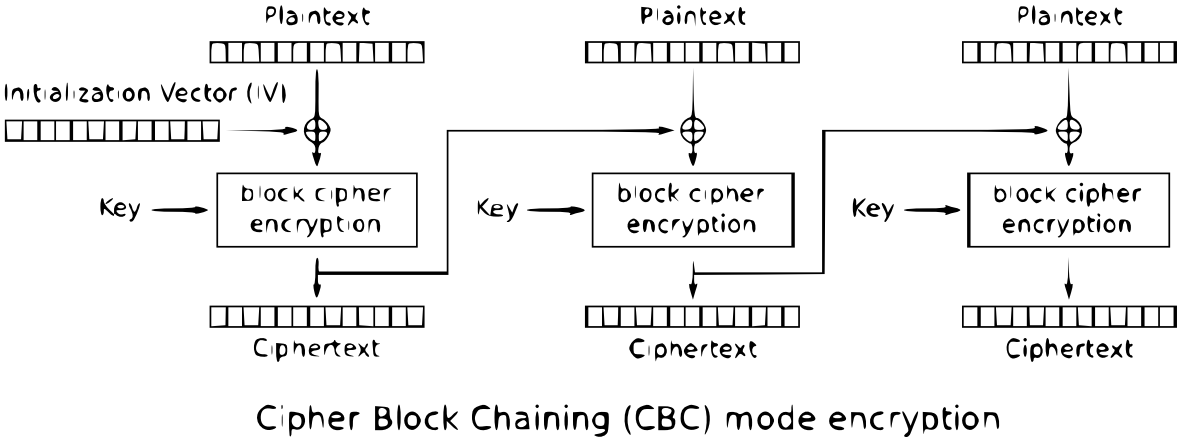


Figura 3.1: Architettura AES CBC

AES GCM

AES GCM è un modo di operazione che usa una modalità contatore per cifrare i dati e una funzione di autenticazione basata sul campo di Galois per garantire l'integrità dei dati. Ogni blocco di testo in chiaro viene sottoposto a un'operazione di XOR con il risultato della cifratura di un contatore, che è un numero che incrementa ad ogni blocco. Il contatore viene inizializzato con un nonce (numero usato una sola volta) e un IV. Il testo cifrato viene poi usato per calcolare un tag di autenticazione, che è un codice che permette di verificare se il testo cifrato è stato alterato o manipolato da un attaccante, il tag in GCM è nella pratica il valore del **polinomio di Galois**. AES GCM ha diversi vantaggi rispetto a AES CBC:

- AES GCM è parallelizzabile, cioè è possibile cifrare o decifrare più blocchi contemporaneamente, perché ogni blocco dipende solo dal contatore. Questo aumenta le prestazioni, soprattutto con dati di grandi dimensioni.
- AES GCM fornisce autenticazione, cioè è possibile verificare se il testo cifrato è stato alterato o manipolato da un attaccante. Questo aumenta la sicurezza e la robustezza dell'algoritmo. Inoltre, AES GCM usa la stessa chiave per cifrare e autenticare i dati, riducendo la complessità e il costo computazionale.

AES GCM è adatto per cifrare dati che richiedono accesso casuale, cioè che vengono letti o scritti in modo non sequenziale, o che vengono trasmessi su una rete.

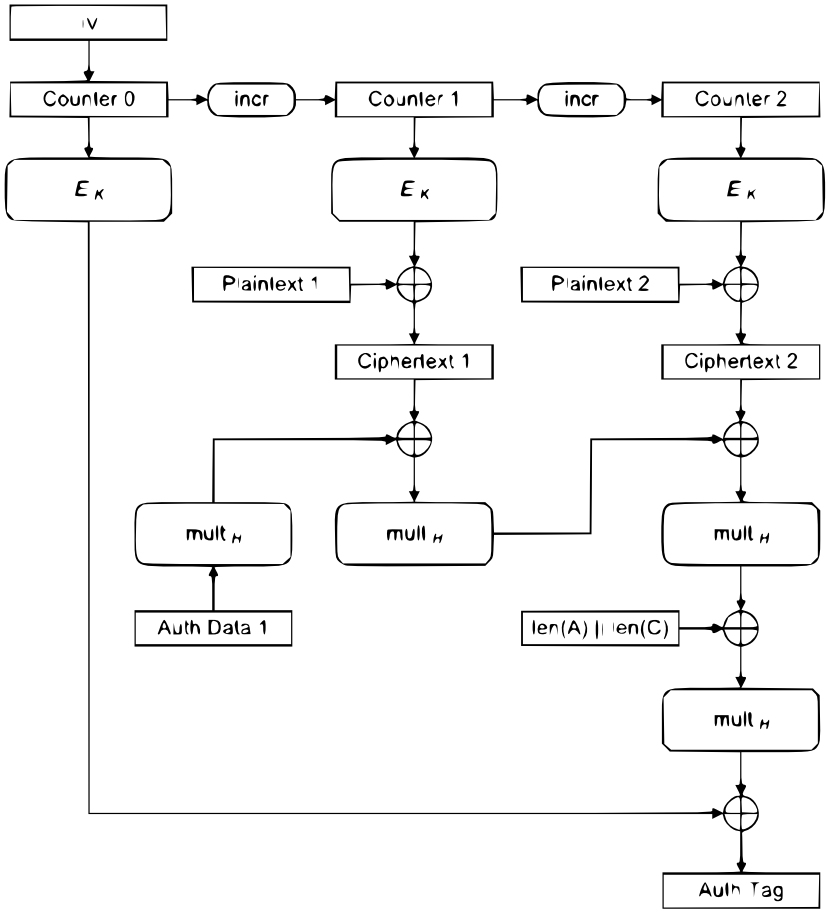


Figura 3.2: Architettura AES GCM

Cos'è il polinomio di Galois

Il **polinomio di Galois** è un concetto fondamentale nella teoria dei campi e nell'algebra astratta. Inizialmente introdotto dal matematico Évariste Galois, il polinomio di Galois è associato a ogni estensione di campo finito. Sia F un campo e E un'estensione di F. Se E è un'estensione di Galois, allora il polinomio minimo di un elemento $a \in E$ su F è detto **polinomio di Galois**. Ad esempio, consideriamo l'estensione $E = \mathbb{Q}(\sqrt{2})$ su $F = \mathbb{Q}$, dove $\sqrt{2}$ è un numero irrazionale. Il polinomio minimo di $\sqrt{2}$ su $\mathbb{Q}$ è $x^2 - 2$. Questo è anche il polinomio di Galois per l'estensione $E/\mathbb{Q}$. La teoria dei polinomi di Galois è profonda e ha numerose

applicazioni in diverse aree della matematica, come la teoria dei numeri, la crittografia e la teoria dei codici. Nel nostro caso il polinomio di Galois è utilizzato per generare un'autenticazione di messaggio durante il processo di cifratura e autenticazione dei dati. Questo polinomio è parte integrante dell'algoritmo di autenticazione del messaggio in GCM, che fornisce un modo efficiente per verificare l'integrità dei dati. Il polinomio di Galois viene utilizzato per moltiplicare i blocchi di dati cifrati con una sequenza di valori di autenticazione derivati da una chiave di autenticazione. Questa operazione produce un'autenticazione dei dati, o MAC, che è inclusa insieme ai dati cifrati per consentire la verifica dell' integrità dei dati.

AES CTR

AES CTR (Counter Mode) è un modo di operazione che utilizza un contatore per cifrare i dati. In questo modo, ogni blocco di testo in chiaro viene sottoposto a un'operazione di XOR con il risultato della cifratura di un contatore, che è un numero che incrementa ad ogni blocco. Il contatore viene inizializzato con un nonce (numero usato una sola volta) e un IV. AES CTR incrementa un contatore per ogni blocco successivo cifrato. Ad esempio, se un'applicazione cifra una stringa venti volte di seguito, allora il risultato di ogni iterazione sarà diverso. Questo perché il contatore viene incrementato.

AES CTR ha diversi vantaggi:

- **Parallelismo**: AES CTR è parallelizzabile, cioè è possibile cifrare o decifrare più blocchi contemporaneamente, perché ogni blocco dipende solo dal contatore. Questo aumenta le prestazioni, soprattutto con dati di grandi dimensioni.
- **Padding**: AES CTR non richiede padding. Normalmente, un algoritmo di cifratura a blocchi (AES, Blowfish, DES, RC2, ecc.) emette un output cifrato che è un multiplo della dimensione del blocco (16 byte per AES come esempio). Con la modalità CTR, il numero di byte di output è esattamente uguale al numero di byte di input, quindi non è richiesto alcun padding.

Questa modalità ha un grande svantaggio; a differenza di AES GCM, AES CTR non fornisce autenticazione. Questo significa che non è possibile verificare se il testo cifrato è stato alterato o manipolato da un attaccante. Per questa ragione, AES CTR è spesso utilizzato in combinazione con un codice di autenticazione del messaggio, come HMAC, per garantire sia la confidenzialità che l'integrità dei dati, tuttavia in TCFS non è stata implementata questa funzionalità. AES CTR è adatto per cifrare dati che richiedono accesso casuale, cioè che vengono letti o scritti in modo non sequenziale, o che vengono trasmessi su una rete.

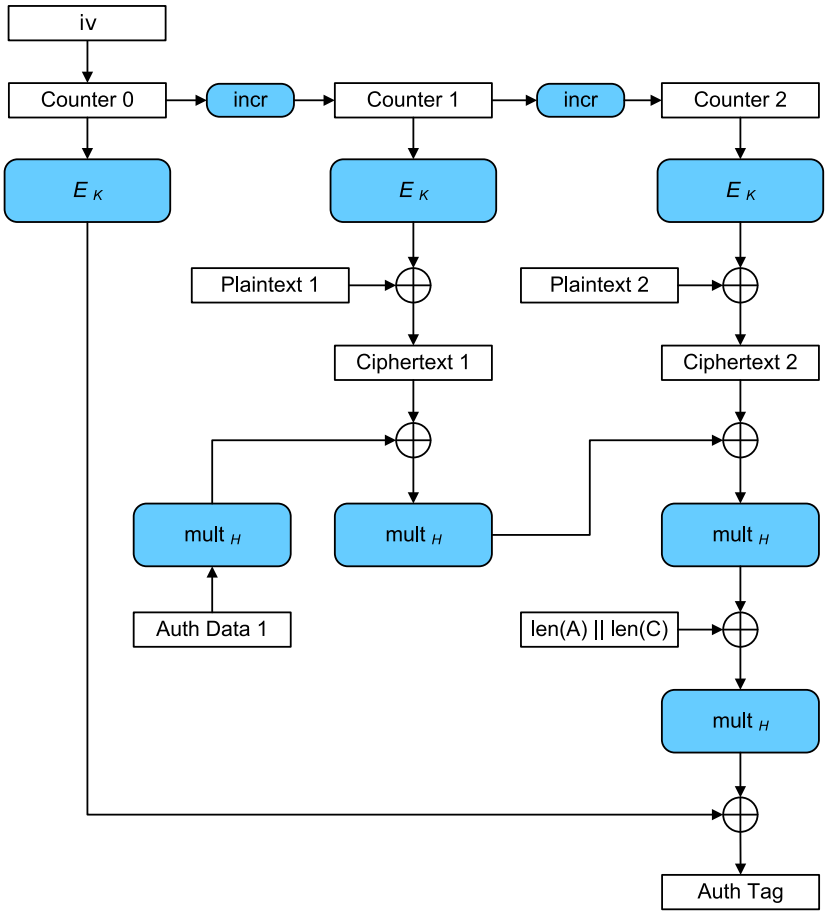


Figura 3.3: Architettura AES CTR

Conclusione

AES CBC, GCM e CTR sono tre modi di operazione di AES che presentano differenze significative in termini di sicurezza, prestazioni e applicazioni. AES GCM è generalmente più sicuro e più veloce di AES CBC, ma richiede una gestione accurata del nonce e dell'IV, che non devono essere riutilizzati per la stessa chiave. AES CBC è più semplice da implementare, ma non è parallelizzabile. AES CTR oltre a essere marginalmente più veloce di GCM offre prestazioni elevate e parallelizzazione, ma richiede una gestione accurata del contatore. Per questi motivi, sia AES GCM che AES CTR sono attualmente disponibili per la cifratura dei file, mentre AES CBC è utilizzato per i nomi delle directory. La scelta tra i tre dipenderà dalle specifiche esigenze dell'utente in termini di sicurezza, prestazioni e gestione delle chiavi.

Sviluppi futuri e limitazioni attuali

Attualmente TCFS non savla un IV per la cifratura dei nomi delle directory, questa limitazione è di caratterizzazione temporale, cioè nonostante lo sviluppo del progetto sia durato diversi mesi non si è riusciti a implementare questa funzionalità. Ci si propone quindi di implementarla quanto prima possibile.

Valutazioni personali

Attraverso l'analisi della situazione attuale si ricava che i benefici dovuti alla diversificazione degli algoritmi nei precedenti due scenari, si intende la cifratura dei nomi e la cifratura dei file stessi, venga surclassata dalla complessità manutentiva maggiorata.

Capitolo 4

Read e Write in TCFS

Nel corso di questo capitolo analizzeremmo in dettaglio il funzionamento delle operazioni di scrittura e lettura all'interno del nostro FS; analizzeremo anche la generazione delle chiavi, i design pattern utilizzati e la logica di base, dando un occhiata al codice di TCFS.

4.1 Cenni generali sulla lettura e scrittura dei file in TCFS

Data la quantità di funzioni crittografice e la complessità di alcune di esse, si è scelto di sviluppare una libreria "ad hoc" per semplificare l'utilizzo delle funzioni di cifratura; la libreria denominata crypt-utils funge a questo scopo, essa espone le seguenti funzioni:

Listing 4.1: Funzioni di crypt-utils

```
1 #define ENCRYPT true
2
3 #define DECRYPT false
4
5 #define IV_SIZE 16 //The fixed size of the initialization
      vector
6
7 extern int do_crypt (int mode, FILE *fp, unsigned char **text,
      int len, unsigned char *key, unsigned char *iv);
8
9 void generate_key (unsigned char *destination);
10
11 unsigned char *encrypt_string (unsigned char *plaintext, const
      char *key, int *encrypted_len);
12
13 unsigned char *decrypt_string (unsigned char *base64_ciphertext
      , const char *key);
14
15 int is_valid_key (const unsigned char *key);
16
17 const char *encrypt_file_name_with_hex (const char *file, const
       char *key);
```

```
18
19 const char *decrypt_file_name_with_hex (const char *enc_file,
       const char *key);
20
21 const char *encrypt_path (const char *path, const char *key);
22
23 const char *encrypt_path_and_filename (const char *path, const
       char *key);
24
25 const char *decrypt_path (const char *encrypted_path, const
       char *key);
26
27 const char *decrypt_path_and_filename (const char *
       encrypted_path, const char *key);
28 extern unsigned char *generate_iv (void);
```

Nel corso delle successive sezioni ci concentreremo sul come i file vengono cifrati, dunque ci interesseremo a do_crypt e generate_key.

4.1.1 La funzione do_crypt

Prima di parlare della cifratura e decifratura dei blocchi è necessario specificare che ambedue le operazioni si ottengono tramite l'utilizzo della funzione do_crypt, che funge da "facade" per le funzioni interne alla libreria:

- **encrypt_file_gcm**: Che si occupa di cifrare il testo in chiaro e scriverlo sul file.
- **decrypt_file_gcm**: Che legge il file cifrato e restituisce il testo in chiaro.

In particolare do_crypt opera in base allo stato della variabile *mode* che può assumere valore ENCRYPT o DECRYPT. Questo permette di modificare facilmente le funzioni ci cifratura e decifratura su file nascondendo gran parte della complessità che esse comportano.

4.1.2 La cifratura dei blocchi

La funzione interna encrypt_file_gcm si occupa della cifratura dei dati e della scrittura di essi su file. La funzione richiede i seguenti parametri:

- **FILE *fp**: Il puntatore al file sul quale scrivere, da notare che TCFS scrive sempre e solo in "*append*"
- **unsigned char *plaintext**: La stringa contenente il testo da cifrare.
- **unsigned char *plaintext_len**: La lunghezza del testo.
- **unsigned char *key**: La chiave AES 256 in formato binario.
- **unsigned char *iv**: L'IV del file.

La funzione si occupa di inizzializzare il contesto di libssl e di cifrare la stringa plaintext, ovviamente la stringa viene cifrata mediante la chiave e l'IV. Dunque per ogni blocco la funzione si comprta nel seguente modo: procede a scrivere prima del blocco la dimensione di quest'ultimo, scrive il testo cifrato e infine scrive scrive il TAG di dimensione definita dalla costante TAG_LEN, che in questa versione è grande esattamente 16 byte. Quest'ultimo ci garantisce l'integrità del blocco scritto poichè al suo interno conserva il valore del polinomio di Galois. Infine ritorna la dimensione del testo cifrato. Di fatto ogni blocco difrato ha la seguente forma:

Lunghezza del testo cifrato (4 byte)	Testo cifrato	Tag (16 byte)

Tabella 4.1: Descrizione dei blocchi nella WRITE

4.1.3 La decifratura dei blocchi

La decifratura viene svolta dalla funzione decrypt_file_gcm che si aspetta i seguenti parametri:

- **FILE *fp**: Il puntatore al file da leggere.
- **unsigned char *key**: La chiave.
- **unsigned char *iv_len**: L' initialization vector.
- **unsigned char **plaintext**: Un puntatore al buffer che deve contenere il testo una volta decifrato.

La funzione si aspetta questa configurazione dei blocchi all'interno del file:

Lunghezza del testo cifrato (4 byte)	Testo cifrato	Tag (16 byte)

Tabella 4.2: Descrizione dei blocchi nella READ

Per decifrare il file la funzione legge la lunghezza del testo cifrato, successivamente legge il testo e il tag e lo decripta tramite la key e l'IV, dunque procede a scrivere il testo in **plaintext*. La funzione ritorna la lunghezza del testo in chiaro.

4.2 L'implementazione della cifratura e della decifratura

TCFS Utilizza AES in modalità GCM per cifrare i blocchi dei file. Questo è una combinazione dell'algoritmo di cifratura simmetrica Advanced Encryption Standard (AES) con la modalità di autenticazione Galois/Counter Mode (GCM), la modalità di funzionamento di questo algoritmo di cifratura è già stato adescritto nel capitolo precedente. Per completezza riportiamo adesso l'implementazione della lettura e della scrittura con AES GCM:

Listing 4.2: Esempio di decrypt e decrypt con gcm

```
1 extern int
```

```
2 decrypt_file_gcm (FILE *fp, unsigned char *key, unsigned char *
    iv,
3                   unsigned char **plaintext, size_t
                       bytes_to_read, off_t offset)
4 {
5   EVP_CIPHER_CTX *ctx = NULL;
6   int len;
7   volatile int plaintext_len = 0;
8   size_t aligned_offset = 0;
9   size_t bytes_read = 0L;
10  size_t bytes_read_prev = 0L;
11  off_t seek_value = 0L;
12  unsigned char *ciphertext = NULL;
13  unsigned char tag[TAG_LEN];
14  *plaintext = NULL;
15
16  if (setjmp (jump_buffer))
17    {
18      logErr (
19          "An error occurred in decrypt_file_gcm! Freeing
               resources... error");
20      if (ctx)
21        EVP_CIPHER_CTX_free (ctx);
22      if (ciphertext)
23        free (ciphertext);
24      return false;
25    }
26
27  // Get file size
28  fseek (fp, 0, SEEK_END);
29  long fsize = ftell (fp);
30  fseek (fp, 0, SEEK_SET); // same as rewind(f);
31
32  // Align to offset
33  if (offset > fsize)
34    {
35      aligned_offset = offset;
36    }
37  while (offset > aligned_offset && offset < fsize)
38    {
39      fread (&seek_value, sizeof (int), 1, fp);
40      bytes_read += (unsigned long)sizeof (int);
41      fseek (fp, seek_value, SEEK_CUR);
42      bytes_read += seek_value;
43      fseek (fp, TAG_LEN, SEEK_CUR);
44      bytes_read += TAG_LEN;
45      aligned_offset += seek_value;
46      if (aligned_offset > offset)
```

```
47          {
48            logDebug ("Offset overshoot");
49            bytes_read = bytes_read_prev;
50            break;
51          }
52        else if (aligned_offset == offset)
53          {
54            logDebug ("Offset Found");
55            break;
56          }
57        else
58          {
59            bytes_read_prev = bytes_read;
60          }
61      }
62
63    fseek (fp, bytes_read, SEEK_SET);
64
65    while (ftell (fp) < fsize && plaintext_len < bytes_to_read)
66      {
67        int ciphertext_len;
68        fread (&ciphertext_len, sizeof (int), 1, fp);
69        if (ciphertext_len < 0){
70            handleErrors();
71          }
72
73        logDebug ("Allocating ciphertext of size %d",
              ciphertext_len);
74        ciphertext = malloc (ciphertext_len);
75        if (!ciphertext)
76          {
77            handleErrors ();
78          }
79
80        fread (ciphertext, sizeof (unsigned char), ciphertext_len
              , fp);
81
82        fread (tag, sizeof (unsigned char), TAG_LEN,
83               fp); // Read the tag from the file
84
85        if (!(ctx = EVP_CIPHER_CTX_new ()))
86          {
87            handleErrors ();
88          }
89
90        if (1 != EVP_DecryptInit_ex (ctx, EVP_aes_256_gcm (),
              NULL, key, iv))
91          {
```

```
92            handleErrors ();
93          }
94
95        if (!EVP_CIPHER_CTX_ctrl (ctx, EVP_CTRL_GCM_SET_TAG,
              TAG_LEN, tag))
96          {
97            handleErrors ();
98          }
99
100       *plaintext = realloc (*plaintext, plaintext_len +
              ciphertext_len);
101       if (!*plaintext)
102         {
103           handleErrors ();
104         }
105
106       if (1
107           != EVP_DecryptUpdate (ctx, *plaintext + plaintext_len
                  , &len,
108                                 ciphertext, ciphertext_len))
109         {
110           handleErrors ();
111         }
112       plaintext_len += len;
113
114       EVP_CIPHER_CTX_free (ctx);
115
116       free (ciphertext);
117     }
118
119   return plaintext_len;
120 }
121
122
123   extern int
124 encrypt_file_gcm (FILE *fp, unsigned char *plaintext, size_t
      plaintext_len,
125                   unsigned char *key, unsigned char *iv, off_t
                          offset)
126 {
127   EVP_CIPHER_CTX *ctx = NULL;
128   int len;
129   int ciphertext_len;
130   unsigned char *ciphertext = NULL;
131   unsigned char tag[TAG_LEN];
132
133   if (setjmp (jump_buffer))
134     {
```

```
135       logErr (
136           "An error occurred in encrypt_file_gcm! Freeing
                resources... error");
137       EVP_CIPHER_CTX_free (ctx);
138       if (ciphertext)
139         free (ciphertext);
140       return false;
141     }
142
143   // Align to offset
144   off_t aligned_offset = (offset / TAG_LEN) * TAG_LEN;
145   fseek (fp, aligned_offset, SEEK_SET);
146
147   if (!(ctx = EVP_CIPHER_CTX_new ()))
148     handleErrors ();
149
150   if (1 != EVP_EncryptInit_ex (ctx, EVP_aes_256_gcm (), NULL,
        key, iv))
151     handleErrors ();
152
153   ciphertext = (unsigned char *)malloc (
154       plaintext_len + EVP_CIPHER_block_size (EVP_aes_256_gcm ()
            ));
155   if (!ciphertext)
156     handleErrors ();
157
158   if (1 != EVP_EncryptUpdate (ctx, ciphertext, &len, plaintext,
        plaintext_len))
159     handleErrors ();
160   ciphertext_len = len;
161
162   if (1 != EVP_EncryptFinal_ex (ctx, ciphertext + len, &len))
163     handleErrors ();
164   ciphertext_len += len;
165
166   if (1 != EVP_CIPHER_CTX_ctrl (ctx, EVP_CTRL_GCM_GET_TAG,
        TAG_LEN, tag))
167     handleErrors ();
168
169   fwrite (&ciphertext_len, sizeof (int), 1, fp);
170
171   if (fwrite (ciphertext, sizeof (unsigned char),
        ciphertext_len, fp)
172       != (unsigned long)ciphertext_len)
173     {
174       handleErrors ();
175     }
176   if (fwrite (tag, sizeof (unsigned char), TAG_LEN, fp)
```

```
177         != (unsigned long)TAG_LEN)
178       {
179         handleErrors ();
180       }
181
182     EVP_CIPHER_CTX_free (ctx);
183     free (ciphertext);
184     return ciphertext_len;
185 }
```

Ora spieghiamo il comportamento delle funzioni:

decrypt_file_gcm

La funzione decrypt_file_gcm è una funzione di decrittografia che utilizza l'algoritmo Galois/Counter Mode (GCM). Questa funzione prende come input un puntatore a un file, una chiave, un vettore di inizializzazione, un puntatore a un puntatore a un array di caratteri, il numero di byte da leggere e un offset. Inizialmente, la funzione inizializza un contesto di cifratura e altre variabili utilizzate per tenere traccia della posizione nel file e della lunghezza del testo in chiaro. Utilizza setjmp per gestire eventuali errori che possono verificarsi durante l'esecuzione della funzione. Se si verifica un errore, la funzione libera le risorse allocate e restituisce 0. Successivamente, la funzione calcola la dimensione del file e si posiziona all'offset specificato mediante un ciclo. Questo viene fatto per calcolare il punto di salto nel file corrispondente all'offset specificato. Entra in un altro ciclo che continua fino a quando non ha letto tutti i byte richiesti o fino a quando non ha raggiunto la fine del file. In ogni iterazione del ciclo, legge la lunghezza del testo cifrato, il testo cifrato stesso e il TAG di autenticazione dal file. Dopo aver letto il testo cifrato e il TAG, la funzione inizializza un nuovo contesto di cifratura e imposta la chiave e il vettore di inizializzazione. Imposta anche il TAG di autenticazione utilizzando EVP_CIPHER_CTX_ctrl. Successivamente, decifra il testo cifrato e lo aggiunge al testo in chiaro. Se si verifica un errore durante la decifratura, la funzione gestisce l'errore e libera le risorse allocate. Infine, alla fine del ciclo, la funzione libera il contesto di cifratura e il testo cifrato e restituisce la lunghezza del testo in chiaro decifrato.

encrypt_file_gcm

La funzione encrypt_file_gcm è utilizzata per crittografare un file utilizzando l'algoritmo di crittografia GCM (Galois/Counter Mode). La funzione inizia inizializzando un contesto di cifratura e alcune variabili. Utilizza setjmp per gestire eventuali errori che possono verificarsi durante l'esecuzione della funzione. Se si verifica un errore, la funzione libera le risorse allocate e restituisce false. Successivamente, la funzione si posiziona all'offset specificato nel file. Crea un nuovo contesto di cifratura e imposta la chiave e il vettore di inizializzazione. Alloca memoria per il testo cifrato. La funzione poi crittografa il testo in chiaro e lo aggiunge al testo cifrato. Se si verifica un errore durante la cifratura, la funzione gestisce l'errore e libera le risorse allocate. Dopo aver cifrato il testo in chiaro, la funzione recupera il tag di autenticazione e scrive la lunghezza del testo cifrato, il testo cifrato e il tag nel file. Infine, la funzione libera

il contesto di cifratura e il testo cifrato e restituisce la lunghezza del testo cifrato. Questa funzione è un esempio di come si può utilizzare l'API OpenSSL per crittografare un file utilizzando l'algoritmo GCM.

4.3 Uso di AES CTR per cifrare e decifrare i file

L'utilizzo di AES CTR è molto simile a GCM con una singola differenza, non viene salvato il TAG dopo il blocco, di fatto i blocchi sul file hanno la seguente forma:

Lunghezza del testo cifrato (4 byte)	Testo cifrato

Tabella 4.3: Descrizione dei blocchi nella READ-WRITE di AES CTR

Le funzioni di lettura e scrittura sono molto simili alla modalità GCM (a meno del tag) e per questo non vengono riportate.

4.3.1 La gerarchia e la generazione delle chiavi in TCFS

Il FS di TCFS non utilizza la stessa chiave per cifrare ogni file. Quando un nuovo file viene creato e successivamente scritto, TCFS genera una nuova chiave di 32 byte, che viene cifrata tramite la chiave principale di TCFS ed inserita tra gli extended attributes del file. Questa chiave viene creata tramite la funzione generate_key riportata di sotto:

Listing 4.3: Esempio di generate_key

```
1  void generate_key (unsigned char *destination)
2  {
3    for (int i = 0; i < 10; i++)
4      {
5        int entropy = check_entropy ();
6        if (entropy < 128)
7          {
8            logWarn ("Not enough entropy, creating some...");
9            add_entropy ();
10         }
11
12       if (RAND_bytes (destination, 32) != true)
13         {
14           logErr ("Cannot generate key");
15           destination = NULL;
16         }
17
18       if (strlen ((const char *)destination) == 32)
19         break;
20     }
21
22   if (!is_valid_key (destination))
```

```
23      {
24        logErr ("ERR: Generated key is invalid\n");
25        destination = NULL;
26      }
27  }
```

La funzione check_entropy() si occupa di verificare la presenza di un numero sufficiente di byte in "/dev/urandom" per la creazione della chiave, mentre add_entropy() affronta un problema più specifico. In particolare, durante la creazione rapida di numerosi file, potrebbe verificarsi una situazione in cui "/dev/urandom" non sia in grado di generare nuovi byte con la stessa velocità. In questo contesto, la funzione add_entropy() interviene attendendo che il sistema accumuli abbastanza entropia per garantire una generazione adeguata di byte.

La differenza tra /dev/random e /dev/urandom

Il dispositivo speciale "/dev/urandom" in ambienti Unix-like fornisce un flusso di dati pseudo-casuali utilizzati per scopi crittografici e di sicurezza. A differenza di "/dev/random", che potrebbe bloccarsi se non c'è abbastanza entropia disponibile, /"dev/urandom" è progettato per essere un generatore di numeri casuali più rapido e non bloccante, tuttavia nel testing eseguito su TCFS si sono verificate due condizioni che hanno mandato in errore la generazione delle chiavi:

- **La velocità di urandom non era sufficente**: Nonostante sia progettato per essere quanto più veloce possibile è stato notato che si poteva cadere nel caso in cui non erano stati generati abbastanza byte per ottenere una nuova chiave.
- **Non bloccante**: In caso si verifichi la condizione precedente allora il comportamento non bloccante forza urandom a restituire una sequenza più piccola di quella richiesta.

Dunque add_entropy() interviene in situazioni in cui la generazione rapida di file potrebbe esaurire temporaneamente l'entropia disponibile, consentendo al sistema di accumulare entropia sufficiente per garantire una generazione affidabile dei byte necessari. Questo approccio garantisce una gestione efficace dell'entropia durante operazioni ad alta intensità di creazione di file.

4.4 Uso di AES CBC per cifrare e decifrare i percorsi

TCFS utilizza AES CBC per cifrare e decifrare i nomi delle cartelle. Abbiamo già discusso ampliamente di CBC dunque concentriamoci sulle funzioni che lo rendono possibile. Si utilizzano le seguenti funzioni della libreria crypt-utils:

Listing 4.4: Funzioni di crypt-utils

```
1 const char *encrypt_path (const char *path, const char *key);
2
3 const char *encrypt_path_and_filename (const char *path, const
      char *key);
```

```
4
5 const char *decrypt_path (const char *encrypted_path, const
      char *key);
6
7 const char *decrypt_path_and_filename (const char *
      encrypted_path,
8                                       const char *key);
```

chiaramente encrypt_path e encrypt_path_and_filename funzionano in maniera estamamente similare, la prima cripta il percorso fino alla cartella desiderata e la seconda cripta il percorso fino al file desiderato, lo stesso vale per decrypt_path e decrypt_path_and_filename.

4.4.1 Perchè AES CBC si scontra con la scrittura dei file in FUSE

Adesso spieghiamo perché si è deciso di non utilizzare CBC per cifrare il contenuto dei file: AES CBC è un modo di operazione per la crittografia a blocchi che presenta alcune sfide quando viene utilizzato con FUSE. In particolare, il problema principale con l'uso di AES CBC in questo contesto è legato alla natura stessa di CBC. In CBC, ogni blocco di testo in chiaro viene combinato con il blocco di testo cifrato precedente prima di essere cifrato. Questo significa che la cifratura di un blocco dipende da tutti i blocchi precedenti. Pertanto, se si desidera modificare un singolo blocco all'interno di un file, non è possibile semplicemente decifrare e ricifrare quel blocco singolo. Invece, è necessario decifrare e ricifrare tutti i blocchi dal blocco modificato fino alla fine del file. Questo può essere molto inefficiente, soprattutto per file di grandi dimensioni. Questo problema è particolarmente evidente quando si utilizza FUSE, che consente operazioni di lettura e scrittura a livello di file. Se un'applicazione tenta di scrivere in una posizione arbitraria all'interno di un file, con AES CBC l'intero file deve essere riscritto, il che può comportare un notevole sovraccarico. Inoltre la lettura randomica in AES CBC è semplicemente impossibile se non si decifra l'intero file. Per questi motivi, AES CBC non è la scelta migliore per la crittografia a livello di directory o filesystem, ed è dunque stato considerato solo per la cifratura dei nomi delle cartelle e dei file.

4.4.2 Dettagli implementativi di encrypt_path_and_filename e decrypt_path_and_fil

Diamo ora un occhiata a encrypt_path_and_filename per capire meglio la sua implementazione:

Listing 4.5: Funzione encrypt_path_and_filename

```
1 const char *encrypt_path_and_filename (const char *path, const
      char *key)
2 {
3   char *result = NULL;
4   char *token, *saveptr;
5
6   // Check if the path is ".", ".."
7   if (strcmp (path, ".") == 0 || strcmp (path, "..") == 0)
8     {
```

```
9         // If it is, no encryption is applied
10        result = strdup (path);
11        if (!result)
12          {
13            logErr ("Cannot allocate memory");
14            exit (EXIT_FAILURE);
15          }
16        return result;
17      }
18    // Check if the path is /
19    else if (strcmp (path, "/") == 0)
20      {
21        logDebug ("got root path\n");
22        result = malloc (1 * sizeof (char));
23        if (result == NULL)
24          {
25            logErr ("Cannot allocate memory");
26            return NULL;
27          }
28        result[0] = '\0';
29      }
30
31    // Copy the original path
32    char *path_copy = strdup (path);
33    if (!path_copy)
34      {
35        logErr ("Cannot allocate memory");
36        exit (EXIT_FAILURE);
37      }
38
39    // Start encryption
40    token = strtok_r (path_copy, "/", &saveptr);
41    while (token != NULL)
42      {
43        // Check if the directory is "." or ".."
44        if (strcmp (token, ".") == 0 || strcmp (token, "..") ==
              0)
45          {
46            // If it is, no encryption is applied
47          }
48        else
49          {
50            // Encrypt each part of the path
51            const char *encrypted_part =
                encrypt_file_name_with_hex (token, key);
52            logDebug ("\tEncrypted %s --> %s\n", token,
                encrypted_part);
53
```

```
54            // Concatenate to the result string
55            if (result == NULL)
56              {
57                // Check if the first character of encrypted_part
                      is '/'
58                if (encrypted_part[0] != '/')
59                  {
60                    result = malloc (strlen (encrypted_part) + 2)
                         ;
61                    if (!result)
62                      {
63                        logErr ("Cannot allocate memory");
64                        exit (EXIT_FAILURE);
65                      }
66                    strcpy (result, "/");
67                    strcat (result, encrypted_part);
68                  }
69                else
70                  {
71                    result = strdup (encrypted_part);
72                  }
73              }
74            else
75              {
76                size_t result_len = strlen (result);
77                size_t encrypted_len = strlen (encrypted_part);
78
79                result = realloc (result, result_len + 1 +
                     encrypted_len + 1);
80                if (!result)
81                  {
82                    logErr ("Cannot allocate memory");
83                    exit (EXIT_FAILURE);
84                  }
85
86                // Check if the first character of encrypted_part
                      is '/'
87                if (encrypted_part[0] != '/')
88                  {
89                    strcat (result, "/");
90                  }
91
92                strcat (result, encrypted_part);
93                logDebug ("Tempresult: %s\n", result);
94              }
95          }
96
97        // Move to the next part of the path
```

```
98          token = strtok_r (NULL, "/", &saveptr);
99      }
100
101     // Free the memory allocated for the path copy
102     free (path_copy);
103
104     return result;
105 }
```

Notiamo innanzitutto che ci sono casi speciali, essa controlla se il percorso del file è uno dei casi speciali ".", ".." o "/". In questi casi, non applica nessuna criptazione e restituisce il percorso originale come risultato, dopo averlo copiato con la funzione strdup. Se il percorso del file non è uno dei casi speciali, la funzione lo copia in una variabile locale path_copy, usando ancora la funzione strdup. La funzione inizia il processo di criptazione, usando la funzione strtok_r per dividere il percorso del file in base al carattere "/". Questo significa che la funzione considera ogni parte del percorso come una directory da criptare separatamente. Per ogni parte del percorso, la funzione controlla se si tratta di una delle directory speciali "." o "..". In questo caso, non applica nessuna criptazione e passa alla parte successiva. Se la parte del percorso non è una directory speciale, la funzione la cripta usando una funzione ausiliaria chiamata encrypt_file_name_with_hex, che prende in input la parte del percorso e la chiave di criptazione. Questa funzione restituisce una stringa che rappresenta la parte criptata in esadecimale. La funzione concatena la parte criptata alla stringa result, aggiungendo il carattere "/" se necessario. Se result è ancora NULL, significa che è la prima parte del percorso da criptare, e quindi la funzione la inizializza con la parte criptata. Altrimenti, la funzione usa la funzione realloc per aumentare la dimensione di result e aggiungere la parte criptata alla fine. La funzione ripete il processo finché non ha esaurito tutte le parti del percorso del file, usando la variabile token per memorizzare la parte corrente e la variabile saveptr per memorizzare il resto del percorso da processare. La funzione libera la memoria allocata per la copia del percorso del file, usando la funzione free e restituisce la stringa result, che contiene il percorso e il nome del file criptati. La funzione decrypt_path_and_filename funziona esattamente allo stesso modo ma invece di invocare la funzione ausiliaria encrypt_file_name_with_hex viene invocata la decrypt_file_name_with_hex.

4.4.3 Le funzioni ausiliarie encrypt_file_name_with_hex e decrypt_file_name_with_

Queste due funzioni fungono da facade per le funzioni encrypt_string e decrypt_string. analizziamole semparatamente.

La funzione encrypt_string

Questa funzione ha lo scopo di inizializzare il contesto di AES CBC, criptare una stringa di lunghezza arbitraria e restituisce la stringa criptata in esadecimale. Diamo anche qui un occhiata alla sua implementazione:

Listing 4.6: Funzione encrypt_string

```
1 unsigned char *encrypt_string (unsigned char *plaintext, const
    char *key,
2                 int *encrypted_key_len)
3 {
4   EVP_CIPHER_CTX *ctx;
5   const EVP_CIPHER *cipher = EVP_aes_256_cbc ();
6   unsigned char iv[AES_BLOCK_SIZE];
7   memset (iv, 0, AES_BLOCK_SIZE);
8
9   ctx = EVP_CIPHER_CTX_new ();
10  if (!ctx)
11    {
12      return NULL;
13    }
14
15  EVP_EncryptInit_ex (ctx, cipher, NULL, (const unsigned char
      *)key, iv);
16
17  size_t plaintext_len = strlen ((const char *)plaintext);
18  unsigned char ciphertext[plaintext_len + AES_BLOCK_SIZE];
19  memset (ciphertext, 0, sizeof (ciphertext));
20
21  int len, total_len = 0;
22  EVP_EncryptUpdate (ctx, ciphertext, &len, plaintext, (int)
      plaintext_len);
23  total_len += len;
24  EVP_EncryptFinal_ex (ctx, ciphertext + total_len, &len);
25  total_len += len;
26  EVP_CIPHER_CTX_free (ctx);
27
28  unsigned char *encoded_string = malloc (total_len * 2 + 1);
29  if (!encoded_string)
30    {
31      logErr ("Cannot allocate memory for encrypt_string
          encoded_string");
32      return NULL;
33    }
34
35  for (int i = 0; i < total_len; i++)
36    {
37      sprintf ((char *)&encoded_string[i * 2], "%02x",
          ciphertext[i]);
38    }
39  encoded_string[total_len * 2] = '\0';
40
41  *encrypted_key_len = total_len * 2;
42  return encoded_string;
43 }
```

Per iniziare, la funzione crea una struttura che contiene le informazioni necessarie per la criptazione, come il tipo di metodo AES, la parola chiave e il valore casuale. Questa struttura si chiama contesto di criptazione e viene inizializzata con le funzioni apposite. Successivamente la funzione calcola la lunghezza della frase da criptare e crea un vettore di byte che ha abbastanza spazio per contenere il codice segreto. Dopo, la funzione inizia a criptare la frase e a scrivere il codice segreto nel vettore di byte, usando le funzioni di criptazione. Queste funzioni prendono il contesto di criptazione, la frase e il vettore di byte e restituiscono il numero di byte scritti. La funzione tiene conto di questo numero per sapere quanto è lungo il codice segreto. Infine, la funzione libera la memoria occupata dal contesto di criptazione e converte il codice segreto in una stringa di caratteri esadecimali. Questo significa che ogni byte viene rappresentato da due cifre da 0 a 9 e da A a F. La funzione restituisce questa stringa, che è il risultato della criptazione.

La funzione decrypt_string

Listing 4.7: Funzione decrypt_string

```
1  unsigned char *
2  decrypt_string (unsigned char *ciphertext, const char *key)
3  {
4    EVP_CIPHER_CTX *ctx;
5    const EVP_CIPHER *cipher = EVP_aes_256_cbc ();
6    unsigned char iv[AES_BLOCK_SIZE];
7    memset (iv, 0, AES_BLOCK_SIZE);
8
9    ctx = EVP_CIPHER_CTX_new ();
10   EVP_DecryptInit_ex (ctx, cipher, NULL, (const unsigned char
       *)key, iv);
11
12   size_t ciphertext_len = strlen ((const char *)ciphertext) /
       2;
13   unsigned char *decoded_ciphertext = malloc (ciphertext_len);
14
15   if (!decoded_ciphertext)
16     {
17       logErr ("Cannot allocate memory for decrypt_string
           decoded_ciphertext");
18       return NULL;
19     }
20
21   for (size_t i = 0; i < ciphertext_len; i++)
22     {
23       char hex[3]
24           = { (char)ciphertext[i * 2], (char)ciphertext[i * 2 +
               1], '\0' };
25       char *endptr;
26       unsigned long byte = strtoul (hex, &endptr, 16);
```

```
27
28        if (*endptr != '\0' || byte > UCHAR_MAX)
29          {
30            logErr ("decrypt string error");
31            free ((void *)decoded_ciphertext);
32            return NULL;
33          }
34        decoded_ciphertext[i] = byte;
35      }
36
37    unsigned char plaintext[ciphertext_len + AES_BLOCK_SIZE + 1];
38    memset (plaintext, 0, sizeof (plaintext));
39
40    int len;
41    EVP_DecryptUpdate (ctx, plaintext, &len, decoded_ciphertext,
42                       (int)ciphertext_len);
43    int padding_len;
44    EVP_DecryptFinal_ex (ctx, plaintext + len, &padding_len);
45    EVP_CIPHER_CTX_free (ctx);
46
47    unsigned char *decrypted_string
48        = (unsigned char *)malloc (len + padding_len + 1);
49
50    if (!decrypted_string)
51      {
52        logErr ("Cannot allocate memory for decrypt_string
              decrypted_string");
53        return NULL;
54      }
55
56    memcpy (decrypted_string, plaintext, len + padding_len);
57    decrypted_string[len + padding_len] = '\0';
58
59    free (decoded_ciphertext);
60
61    return decrypted_string;
62  }
```

Per decriptare il codice segreto, la funzione deve prima convertirlo in un vettore di byte, cioè una sequenza di byte che può essere manipolata da libssl. Per fare questo, la funzione usa un ciclo for che scorre la stringa esadecimale e prende ogni coppia di caratteri. Poi, usa la funzione strtoul per convertire la coppia di caratteri in un valore numerico, interpretando la stringa come un numero esadecimale. Se la conversione ha successo, la funzione assegna il valore numerico al byte corrispondente nel vettore di byte. Se la conversione fallisce o il valore numerico è troppo grande per essere contenuto in un byte, la funzione restituisce un errore e termina. Dopo aver convertito il codice segreto in un vettore di byte, la funzione deve creare una struttura che contiene le informazioni necessarie per la decrittazione, come il tipo di metodo AES, la parola

chiave e il valore casuale. Questa struttura si chiama contesto di decrittazione e viene usata dalle funzioni di decrittazione per eseguire le operazioni inverse di quelle fatte dalla criptazione. Il tipo di metodo AES è una costante che indica il tipo di cifrario AES da usare (in questo caso AES-256-CBC, che significa che usa una chiave di 256 bit e una modalità di operazione a blocchi concatenati). La parola chiave è una stringa che rappresenta la chiave segreta di decrittazione, che deve essere la stessa usata per la criptazione. Il valore casuale è un vettore di byte che rappresenta il vettore di inizializzazione, che è un valore usato per rendere più sicura la decrittazione. In questo caso, il vettore di inizializzazione è inizializzato con tutti zeri, usando la funzione memset. Per inizializzare il contesto di decrittazione, la funzione usa due funzioni apposite: EVP_CIPHER_CTX_new e EVP_DecryptInit_ex. La prima funzione crea un nuovo contesto di decrittazione e restituisce un puntatore alla struttura. La seconda funzione inizializza il contesto di decrittazione con il tipo di metodo AES, la parola chiave e il vettore di inizializzazione. Dopo aver inizializzato il contesto di decrittazione, la funzione deve creare un vettore di byte che ha abbastanza spazio per contenere la frase decrittata. Il vettore di byte ha una dimensione pari alla lunghezza del codice segreto diviso due (perché ogni byte corrisponde a due caratteri esadecimali) più la dimensione di un blocco AES (16 byte), per garantire che ci sia abbastanza spazio per contenere la frase decrittata. Il vettore di byte viene riempito con dei valori nulli, usando la funzione memset, per evitare che ci siano dei dati inutili. Poi, la funzione inizia a decrittare il codice segreto e a scrivere la frase nel vettore di byte, usando le funzioni di decrittazione. Queste funzioni sono EVP_DecryptUpdate e EVP_DecryptFinal_ex. La prima funzione decritta una parte del codice segreto e la scrive nel vettore di byte, restituendo il numero di byte scritti. La seconda funzione decritta la parte finale del codice segreto e la scrive nel vettore di byte, restituendo il numero di byte scritti. La funzione tiene conto di questi numeri per sapere quanto è lunga la frase decrittata. Infine, la funzione libera la memoria occupata dal contesto di decrittazione, usando la funzione EVP_CIPHER_CTX_free, e copia la frase dal vettore di byte in una stringa di caratteri, usando la funzione malloc per allocare la memoria e la funzione memcpy per copiare i dati. Questa stringa viene terminata con il carattere '\0', che indica la fine della frase. La funzione restituisce questa stringa, che è il risultato della decrittazione.

Capitolo 5

Analisi delle prestazioni

In questo capitolo andiamo ad analizzare le prestazioni di TCFS, comparandola con altri FS.

5.1 Il Server per il Testing

Come precedentemente menzionato, TCFS è progettato principalmente per essere utilizzato in combinazione con un filesystem di rete. Per tutti i test, utilizzeremo NFS, che interagirà con BTFS, il filesystem utilizzato dal server remoto. Ecco come NFS interagisce con BTFS:

- **Montaggio del Filesystem**: Un client NFS, per accedere ai file su un server NFS, deve prima "montare" il filesystem remoto. Questo processo associa un punto di montaggio locale (una directory o cartella) al filesystem remoto.
- **Richiesta di File**: Quando un'applicazione sul client NFS richiede un file che risiede sul filesystem NFS montato, il client NFS invia una richiesta al server NFS. Questa richiesta include il percorso del file come appare nel punto di montaggio locale.
- **Interazione con il Filesystem Sottostante**: Il server NFS riceve la richiesta e la traduce in un'operazione sul suo filesystem locale. Ad esempio, se la richiesta era di leggere un file, il server NFS apre il file sul suo filesystem locale e legge i dati richiesti.
- **Risposta al Client**: Il server NFS invia una risposta al client NFS con i dati richiesti (se la richiesta era di leggere un file) o con una conferma dell'operazione (se la richiesta era di scrivere su un file o di modificare i metadati di un file).
- **Visualizzazione dei Dati**: Il client NFS riceve la risposta e la presenta all'applicazione che ha fatto la richiesta come se i dati fossero stati letti da un file locale.

Per quanto riguarda le specifiche del server, abbiamo utilizzato una macchina remota equipaggiata con 12 dischi Seagate BarraCuda da 1 TB (ST1000DM010) ciascuno, da 6 GBit/s, 3,5", 7.200 RPM, con 64 MB di cache, configurati in RAID 10 (Mirror + Stripe) che ci garantisce una velocità di 12x in lettura e 6x in scrittura, fornendo così circa 6 TB di spazio. La macchina dispone di 10 core a 4.6 GHz, 64 GB di RAM (in quad channel) e una connessione Gigabit.

5.2 La macchina client

Tutti questi test verranno eseguiti su una macchina client Huawei D15 2020, con un processore Ryzen 5 3500U, con 4 core, 8 thread, un Max. Boost Clock fino a 3.7GHz, un TDP di 15 Watt e con 8GB di RAM. Tutti i test verranno svolti su OS Fedora 39, utilizzando la versione del kernel Linux 6.7.4-200.fc39.x86_64. Tutte le operazioni di lettura e scrittura verranno eseguite sul server tramite NFS con una velocità di connessione di 500 Mb/s.

5.3 Criteri e metodi di valutazione

Per il benchmarking della velocità del filesystem, si prevede di testare sia la lettura che la scrittura sequneziale e randomica. Verranno valutati i Mb/s, il carico sulla CPU complessivo e il throughput.

5.3.1 Lettura e Scrittura

Le operazioni di lettura e scrittura saranno sottoposte a un'analisi dettagliata, utilizzando file e blocchi di dimensioni variabili. Prevediamo di testare file che variano da 1MB a 100MB e blocchi che vanno da 1KB a 16MB. Queste dimensioni non sono state scelte a caso, ma sono state selezionate "*cum grano salis*". Questo perché le dimensioni dei blocchi possono avere un impatto significativo sulle prestazioni del filesystem: blocchi più piccoli possono portare a un utilizzo più efficiente dello spazio su disco, ma possono anche causare un overhead maggiore a causa del numero maggiore di operazioni di I/O necessarie. D'altra parte, blocchi più grandi possono migliorare le prestazioni di lettura e scrittura sequenziali, ma possono portare a uno spreco di spazio su disco se i file sono significativamente più piccoli della dimensione del blocco.

Per ogni file, testeremo le operazioni con almeno tre dimensioni diverse per i blocchi. Le operazioni di lettura e scrittura verranno eseguite sia in sequenza che in modo casuale, registrando le metriche di ciascuna operazione. Questo processo sarà ripetuto più volte per ottenere una media affidabile. In questo modo, saremo in grado di comprendere meglio come le dimensioni dei blocchi influenzano le prestazioni del filesystem in vari scenari di utilizzo.

5.3.2 Il programma di testing

Si è deciso di sviluppare un tool di benchmarking personalizzato, specificamente progettato per il testing dei filesystem. Questo strumento è stato implementato in Python, un linguaggio noto per la sua leggibilità e flessibilità. La scelta di Python non è casuale: la sua sintassi intuitiva e la sua struttura semplificata rendono il codice facilmente accessibile, quasi come se fosse stato progettato con l'utente meno brillante in mente. Il codice del tool, che verrà presentato di seguito, è commentato. Questo approccio, che evita di sovraccaricare il lettore con dettagli tecnici superflui, facilita la comprensione del suo funzionamento. Il tool di benchmarking è in grado di raccogliere una vasta gamma di metriche. Tuttavia, per lo scopo di questo studio, abbiamo scelto di concentrarci solo su un sottoinsieme di queste metriche, ritenute più rilevanti per la nostra analisi. Questa scelta riflette una filosofia ben nota nel mondo Python: "There should be one– and preferably only one –obvious way to do it". Tuttavia,

è interessante notare come questa filosofia, pur essendo un pilastro della comunità Python, non sempre trovi una perfetta corrispondenza nella realtà. Infatti, chiunque abbia avuto a che fare con Python sa che, nonostante la sua apparente semplicità, il linguaggio può riservare sorprese. Questo non è un giudizio sul linguaggio in sé, ma piuttosto un invito a considerare che ogni strumento ha i suoi punti di forza e le sue sfide. E Python, nonostante tutte le sue virtù, non fa eccezione. In effetti, potrebbe quasi sembrare che Python, con la sua promessa di un solo modo ovvio per fare le cose, stia in qualche modo giocando un brutto scherzo ai suoi utenti. Ma, come si suol dire, "ogni rosa ha le sue spine".

Listing 5.1: Script Python per il benchmarking

```
import os
import time
import random
import threading
import ctypes
import psutil

# Define constants for posix_fadvise
POSIX_FADV_NORMAL = 0
POSIX_FADV_RANDOM = 1
POSIX_FADV_SEQUENTIAL = 2
POSIX_FADV_WILLNEED = 3
POSIX_FADV_DONTNEED = 4
POSIX_FADV_NOREUSE = 5

# Fetch a reference to posix_fadvise
libc = ctypes.CDLL('libc.so.6')
posix_fadvise = libc.posix_fadvise
posix_fadvise.argtypes = [ctypes.c_int, ctypes.c_int64, ctypes.
    c_int64, ctypes.c_int]
posix_fadvise.restype = ctypes.c_int

# Function to dump the system cache for a specific file
def drop_cache(file_name):
    fd = os.open(file_name, os.O_RDONLY)
    try:
        res = posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED)
        if res != 0:
            raise OSError(res, os.strerror(res))
    finally:
        os.close(fd)

# Function to convert size from string to bytes
def convert_size(size):
    if size.endswith('K'):
        return int(size[:-1]) * 1024
    elif size.endswith('M'):
```

```
37          return int(size[:-1]) * 1024 * 1024
38      elif size.endswith('G'):
39          return int(size[:-1]) * 1024 * 1024 * 1024
40      else:
41          return int(size)
42
43  # File size
44  file_size = convert_size('100M')
45  # Block size
46  block_size = convert_size('16M')
47  # Shared variables
48  blocks_processed = 0
49  total_blocks = file_size // block_size
50  # Generate a block of random data once
51  random_data = bytearray(os.urandom(block_size))
52  stop_thread = False
53
54  def change_random_data():
55      global random_data, stop_thread
56      while not stop_thread:
57          # Read a random byte from /dev/urandom
58          try:
59              with open('/dev/urandom', 'rb') as f:
60                  while True:
61                      random_byte = f.read(block_size)
62                      if len(random_byte) != block_size:
63                          # If no data is read, wait and try
                                again
64                          time.sleep(0.5)
65                          continue
66                      else:
67                          # Change a random byte in the block
68                          index = random.randint(0, block_size -
                                1)
69                          random_data[index] = random_byte[0]
70                          break
71          except IOError:
72              print("Could not read from /dev/urandom.")
73
74  # Start the thread that changes random_data
75  change_thread = threading.Thread(target=change_random_data)
76  change_thread.start()
77
78  def write_sequential(file_name):
79      global blocks_processed
80      start_time = time.time()
81      with open(file_name, 'wb') as f:
82          for _ in range(total_blocks):
```

```
83              f.write(random_data)  # Use the pre-generated
                    random data
84              blocks_processed += 1
85      end_time = time.time()
86      elapsed_time = end_time - start_time
87      speed = file_size / elapsed_time / (1024 * 1024)  # Convert
            to MB/s
88      return elapsed_time, speed
89
90  def read_sequential(file_name):
91      global blocks_processed
92      start_time = time.time()
93      with open(file_name, 'rb') as f:
94          while f.read(block_size):
95              blocks_processed += 1
96      end_time = time.time()
97      elapsed_time = end_time - start_time
98      speed = file_size / elapsed_time / (1024 * 1024)  # Convert
            to MB/s
99      return elapsed_time, speed
100
101 def write_random(file_name):
102     global blocks_processed
103     start_time = time.time()
104     with open(file_name, 'wb') as f:
105         for _ in range(total_blocks):
106             f.seek(random.randint(0, file_size - block_size))
107             f.write(random_data)  # Use the pre-generated
                    random data
108             blocks_processed += 1
109     end_time = time.time()
110     elapsed_time = end_time - start_time
111     speed = file_size / elapsed_time / (1024 * 1024)  # Convert
            to MB/s
112     return elapsed_time, speed
113
114 def read_random(file_name):
115     global blocks_processed
116     start_time = time.time()
117     with open(file_name, 'rb') as f:
118         for _ in range(total_blocks):
119             f.seek(random.randint(0, file_size - block_size))
120             f.read(block_size)
121             blocks_processed += 1
122     end_time = time.time()
123     elapsed_time = end_time - start_time
124     speed = file_size / elapsed_time / (1024 * 1024)  # Convert
            to MB/s
```

```
125     return elapsed_time, speed
126
127 def measure_cpu_usage(func, *args, **kwargs):
128     # Get the initial CPU time
129     initial_cpu_time = psutil.cpu_times()
130
131     # Execute the function and get the result
132     result = func(*args, **kwargs)
133
134     # Get the final CPU time
135     final_cpu_time = psutil.cpu_times()
136
137     # Calculate the CPU time used
138     user_time_used = final_cpu_time.user - initial_cpu_time.
            user
139     system_time_used = final_cpu_time.system - initial_cpu_time
            .system
140     cpu_time_used = user_time_used + system_time_used
141
142     return result, cpu_time_used
143
144 def measure_iops(func, *args, **kwargs):
145     # Get the start time
146     start_time = time.time()
147
148     # Execute the function and get the result
149     result = func(*args, **kwargs)
150
151     # Get the end time
152     end_time = time.time()
153
154     # Calculate the IOPS
155     iops = total_blocks / (end_time - start_time)
156
157     return result, iops
158
159 def measure_throughput(func, *args, **kwargs):
160     # Get the start time
161     start_time = time.time()
162
163     # Execute the function and get the result
164     result = func(*args, **kwargs)
165
166     # Get the end time
167     end_time = time.time()
168
169     # Calculate the throughput
170     throughput = file_size / (end_time - start_time)
```

```
171
172     return result, throughput
173
174 monitor = True
175 cpu_load = 0.0
176 def cpu_load_monitor():
177     global cpu_load
178     cpu_load_sum = 0.0
179     cpu_load_count = 0
180     while monitor:
181         cpu_load_sum += psutil.cpu_percent(interval=1)
182         cpu_load_count += 1
183     cpu_load = cpu_load_sum / cpu_load_count if cpu_load_count
        else 0
184
185 def measure_cpu_load(func, *args, **kwargs):
186     global monitor
187     monitor = True
188     monitor_thread = threading.Thread(target=cpu_load_monitor)
189     monitor_thread.start()
190
191     # Execute the function and get the result
192     result = func(*args, **kwargs)
193
194     monitor = False
195     monitor_thread.join()
196
197     return result, cpu_load
198
199 def main():
200     global blocks_processed, stop_thread, monitor
201     file_name = '/mnt/B/tcfsBenchmark'
202     print('Creating a test file of size {} Mb...'.format((
        file_size / 1024 / 1024)))
203     with open(file_name, 'wb') as f:
204         for _ in range(file_size // block_size):  # Corrected
            here
205             f.write(random_data)  # Use the pre-generated
                random data
206
207     # List of functions to execute
208     functions = [write_sequential, read_sequential,
        write_random, read_random]
209
210     for func in functions:
211         print (f"Executing function {func.__name__}")
212         # Reset blocks_processed for each function
213         blocks_processed = 0
```

```
214         # Start the function in a separate thread
215         thread = threading.Thread(target=func, args=(file_name
                ,))
216         thread.start()
217         # Monitor progress
218         while thread.is_alive():
219             progress = blocks_processed / total_blocks * 100
220             print('Progress: {:.2f}%'.format(progress), end='\r
                    ', flush=True)  # Added flush=True here
221         thread.join()  # Wait for the thread to finish before
                moving on to the next function
222         (elapsed_time, speed), cpu_time_used =
                measure_cpu_usage(func, file_name)
223         _, iops = measure_iops(func, file_name)
224         _, throughput = measure_throughput(func, file_name)
225         _, cpu_load = measure_cpu_load(func, file_name)
226         print('\n{}: elapsed time = {:.2f} seconds, speed =
                {:.2f} MB/s, CPU time used = {:.2f} seconds, CPU
                load = {:.2f}, IOPS = {:.2f}, Throughput = {:.2f}
                bytes/second'.format(func.__name__, elapsed_time,
                speed, cpu_time_used, cpu_load, iops, throughput))
227         drop_cache(file_name=file_name)
228
229     # Stop the change_random_data thread
230     stop_thread = True
231     change_thread.join()
232
233     os.remove(file_name)
234
235 if __name__ == "__main__":
236     main()
```

5.4 Valutazioni sulla velocità della rete e performance di TCFS

Un filesystem crittografico di rete può effettivamente avere un impatto minore su una rete più lenta rispetto a una più veloce. Questo perché il tempo necessario per cifrare i dati può essere "nascosto" o mitigato dalla lentezza della rete stessa. Per spiegare meglio, consideriamo il seguente scenario: quando si trasmettono dati su una rete, ci sono due principali fattori che contribuiscono al tempo totale di trasmissione. Il primo è il tempo necessario per preparare i dati per la trasmissione, che include operazioni come la cifratura. Il secondo è il tempo necessario per trasmettere effettivamente i dati sulla rete. In una rete veloce, il tempo di trasmissione dei dati è breve, quindi il tempo di cifratura può costituire una parte significativa del tempo totale di trasmissione. In altre parole, la cifratura può essere il collo di bottiglia che rallenta l'intero processo. Tuttavia, in una rete lenta, il tempo di trasmissione dei dati è lungo. Pertanto, il tempo necessario per cifrare i dati può essere "assorbito" nel tempo di trasmissione. In altre parole, mentre i dati vengono cifrati, i dati già cifrati possono iniziare a

essere trasmessi. Di conseguenza, il tempo totale di trasmissione potrebbe non essere molto più lungo rispetto a una situazione in cui i dati non vengono cifrati. Quando si utilizza un filesystem crittografico di rete su una rete geografica, il tempo necessario per cifrare i dati può essere ancora più facilmente "nascosto" nel tempo di trasmissione dei dati sulla rete, a causa delle latenze più elevate. In sintesi, in una rete lenta, il tempo necessario per la cifratura dei dati può essere meno evidente perché è nascosto dal tempo di trasmissione dei dati sulla rete. Questo può portare a una percezione di minori perdite di prestazioni dovute alla cifratura in una rete lenta rispetto a una veloce.

5.5 Risultati e discussione

In questa sezione sono riportati i risultati di lettura e scrittura per i vari FS testati. Analizziamoli uno a uno.

5.5.1 TCFS utilizzando AES 256 CTR con NFS

Analizziamo il comportamento di TCFS in modalità AES 256 CTR, ricordiamo che nella versione attuale questo algoritmo non è parallelizzato.

File (MB)	**Blocco (KB)**	**Operazione**	**MB/s**	**Utilizzo CPU (%)**	**Throughput (b/s)**
1	4	write sequential	2.28	27.10	2398819.75
1	4	read sequential	7.72	22.80	7247444.57
1	4	write random	1.33	34.80	1458171.51
1	4	read random	0.14	45.68	194777.08
1	8	read random	4.33	23.40	4586300.38
1	8	read sequential	12.44	21.20	17569065.15
1	8	write random	3.06	26.40	3301606.20
1	8	read random	1.11	41.70	727779.51
1	16	read random	8.57	21.40	8819553.03
1	16	read sequential	18.86	24.40	21070605.96
1	16	write random	6.65	22.60	6285015.60
1	16	read random	2.64	30.70	4311260830.58
5	4	read random	2.23	35.30	2373715.93
5	4	read sequential	2.31	48.67	2516080.90

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
5	4	write random	1.42	39.08	1469180.80
5	4	read random	0.03	54.18	47170.89
5	8	read random	4.20	32.75	4096254.93
5	8	read sequential	4.30	42.70	4691622.94
5	8	write random	2.85	40.40	2939063.21
5	8	read random	0.22	52.71	144792.49
5	16	read random	9.71	28.50	9565724.43
5	16	read sequential	8.09	2.53	8307699.96
5	16	write random	6.41	34.40	6171435.72
5	16	read random	0.66	45.59	693149.29
10	8	read random	4.03	36.97	4271174.68
10	8	read sequential	2.56	17.61	2774961.88
10	8	write random	2.86	40.23	2983616.54
10	8	read random	0.07	52.08	108989.75
10	1024	read random	59.66	23.00	56716713.60
10	1024	read sequential	82.41	25.10	87096583.73
10	1024	write random	54.12	21.50	50098321.88
10	1024	read random	85.32	24.70	87764688.38
10	8192	read random	79.82	21.90	74568817.93
10	8192	read sequential	100.39	22.90	106603286.56
10	8192	write random	78.26	21.60	71662802.40
10	8192	read random	102.74	23.50	120627941.92
50	1024	read random	46.19	29.60	45275246.61
50	1024	read sequential	34.96	48.80	34852774.43
50	1024	write random	41.24	30.45	41601704.60
50	1024	read random	29.75	52.40	33762252.01

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
50	8192	read random	59.83	33.10	57801131.92
50	8192	read sequential	42.71	40.55	43941611.94
50	8192	write random	66.43	33.90	70979325.37
50	8192	read random	78.67	52.50	52438626.84
50	16384	read random	57.50	33.80	58918321.84
50	16384	read sequential	45.77	42.30	45382606.11
50	16384	write random	56.72	37.70	59715774.81
50	16384	read random	59.60	54.20	52602674.73
100	1024	read random	44.56	31.87	47557147.84
100	1024	read sequential	21.66	55.76	23472142.77
100	1024	write random	41.40	34.93	43735567.80
100	1024	read random	16.12	55.79	16079483.55
100	8192	read random	48.91	32.50	50039793.74
100	8192	read sequential	24.86	57.80	26472049.87
100	8192	write random	47.43	32.63	48935023.71
100	8192	read random	38.49	51.95	35320372.48
100	16384	read random	49.75	33.50	47432308.69
100	16384	read sequential	25.16	58.72	23021673.76
100	16384	write random	47.65	33.47	47023893.84
100	16384	read random	30.49	70.57	31715630.99

5.5.2 BTFS su NFS senza criptazione

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
1	4	write sequential	55.47	19.20	50867402.77
1	4	read sequential	452.61	20.30	493441771.69

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
1	4	write random	23.99	18.30	22775532.93
1	4	read random	172.86	18.50	210876798.58
1	8	write sequential	49.83	18.60	49974961.78
1	8	read sequential	499.23	18.80	1257662714.07
1	8	write random	27.92	18.70	36144366.46
1	8	read random	392.39	18.50	650213854.39
1	16	write sequential	68.82	18.60	86431099.76
1	16	read sequential	509.23	8.42	1363735352.28
1	16	write random	57.83	18.20	66401644.34
1	16	read random	492.78	14.89	684307843.64
5	4	write sequential	49.14	20.30	49752895.56
5	4	read sequential	502.34	11.38	816570091.18
5	4	write random	20.93	20.90	22305204.35
5	4	read random	429.22	19.80	501636347.28
5	8	write sequential	42.73	24.20	46245476.04
5	8	read sequential	502.34	12.74	822218454.12
5	8	write random	34.65	20.30	32675276.79
5	8	read random	497.56	13.85	1041352112.30
5	16	write sequential	75.12	21.20	71285995.34
5	16	read sequential	501.23	18.20	1808407282.53
5	16	write random	41.75	19.80	49662556.90
5	16	read random	498.78	19.00	1897345345.60

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
10	8	write sequential	50.37	21.30	58093764.18
10	8	read sequential	502.34	19.00	867464795.09
10	8	write random	28.88	22.50	31761131.89
10	8	read random	497.56	18.60	1041771445.41
10	1024	write sequential	97.79	19.50	84955378.83
10	1024	read sequential	503.45	19.90	4779446328.09
10	1024	write random	74.86	19.90	97034211.17
10	1024	read random	496.34	18.60	4063235875.00
10	8192	write sequential	130.94	20.30	116068249.71
10	8192	read sequential	504.56	19.40	3609690176.55
10	8192	write random	104.69	21.10	130952312.52
10	8192	read random	495.12	19.10	3994229871.13
50	1024	write sequential	91.06	24.70	92609442.68
50	1024	read sequential	505.67	19.70	8135792132.72
50	1024	write random	85.92	25.20	107060841.25
50	1024	read random	493.90	18.70	6130877817.42
50	8192	write sequential	96.02	25.00	107843787.33
50	8192	read sequential	506.78	19.80	3841692590.19
50	8192	write random	104.33	24.70	107826654.21
50	8192	read random	492.46	18.60	4497531917.11
50	16384	write sequential	93.81	25.60	104003956.53
50	16384	read sequential	507.89	18.40	4194128007.38

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
50	16384	write random	106.17	22.80	107233072.47
50	16384	read random	491.02	18.50	4960575807.70
100	1024	write sequential	90.52	25.40	91030534.46
100	1024	read sequential	508.99	19.10	7545372136.81
100	1024	write random	83.85	25.30	92267923.06
100	1024	read random	489.58	17.30	6069787340.40
100	8192	write sequential	97.79	31.80	94533707.66
100	8192	read sequential	509.14	19.50	5009335753.05
100	8192	write random	102.02	24.65	113555411.54
100	8192	read random	488.14	20.50	4311260830.58
100	16384	write sequential	100.46	25.55	94192276.81
100	16384	read sequential	509.14	18.80	5050987689.76
100	16384	write random	108.86	32.80	113611380.63
100	16384	read random	486.70	19.10	8131279601.90

5.5.3 PTFS

Si ricorda che PTFS non supporta la parallelizzazione

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
1	4	write sequential	30.53	19.40	25810737.99
1	4	read sequential	233.63	18.60	265006417.88
1	4	write random	13.37	20.30	11811529.17
1	4	read random	156.98	19.30	153034082.99
1	8	write sequential	33.05	23.80	28881503.76

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
1	8	read sequential	496.54	19.30	665765442.19
1	8	write random	19.68	19.80	17245956.23
1	8	read random	149.75	18.90	244146025.93
1	16	write sequential	29.06	24.90	38643082.55
1	16	read sequential	481.22	19.00	505755118.57
1	16	write random	25.57	20.70	19891842.13
1	16	read random	284.11	23.30	339565048.73
5	4	write sequential	29.29	22.60	32446509.65
5	4	read sequential	499.87	22.10	561634381.05
5	4	write random	11.30	22.70	11990187.97
5	4	read random	147.69	21.10	148602733.85
5	8	write sequential	28.42	21.80	27736447.95
5	8	read sequential	499.92	20.00	830070683.81
5	8	write random	16.52	22.90	17220215.61
5	8	read random	211.75	19.50	224175103.02
5	16	write sequential	34.45	20.70	27910353.45
5	16	read sequential	499.76	19.50	863750836.86
5	16	write random	24.73	21.90	22072230.89
5	16	read random	254.31	21.80	333150007.66
10	8	write sequential	29.24	27.20	28515914.16
10	8	read sequential	499.84	18.20	799499456.66
10	8	write random	17.48	24.70	18477424.68

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
10	8	read random	217.45	19.70	228216243.30
10	1024	write sequential	31.32	25.70	35022818.83
10	1024	read sequential	499.91	18.20	883212810.49
10	1024	write random	34.85	23.20	35886571.82
10	1024	read random	499.89	18.40	1253290354.24
10	8192	write sequential	40.42	23.90	43319197.56
10	8192	read sequential	402.68	20.00	893403451.51
10	8192	write random	45.28	23.60	43384749.12
10	8192	read random	495.32	18.30	1089704289.17
50	1024	write sequential	31.39	29.65	36199784.74
50	1024	read sequential	504.67	19.20	940179421.26
50	1024	write random	30.31	28.10	33734102.54
50	1024	read random	504.12	22.20	974869666.56
50	8192	write sequential	35.30	29.50	32114601.56
50	8192	read sequential	503.89	19.70	1024655425.65
50	8192	write random	33.69	28.20	36443371.02
50	8192	read random	504.57	18.20	1317877308.39
50	16384	write sequential	35.05	29.75	35600309.01
50	16384	read sequential	504.23	20.80	922037793.32
50	16384	write random	36.36	26.90	34183453.23
50	16384	read random	504.45	19.10	1338419510.38
100	1024	write sequential	31.12	29.15	33501250.50

File (MB)	Blocco (KB)	Operazione	MB/s	Utilizzo CPU (%)	Throughput (b/s)
100	1024	read sequential	504.33	20.60	1039245765.82
100	1024	write random	35.20	29.30	35365136.03
100	1024	read random	504.15	21.50	1149032035.95
100	8192	write sequential	33.37	30.73	38188489.47
100	8192	read sequential	495.67	21.30	1050523108.90
100	8192	write random	38.09	30.97	38085067.97
100	8192	read random	495.23	20.30	1350320079.31
100	16384	write sequential	30.89	30.90	35045035.83
100	16384	read sequential	496.12	21.90	1036803179.47
100	16384	write random	38.10	32.00	39262702.02
100	16384	read random	496.78	21.10	1179157844.37

5.6 Valutazione dei risultati

Dai risultati dei test, è evidente che il costo prestazionale del TCFS è significativo. Tuttavia, è possibile mitigare questo costo attraverso l'implementazione della parallelizzazione del processo di cifratura, a condizione che le dimensioni dei blocchi siano sufficientemente grandi. Questo perché, nel caso di blocchi di dimensioni ridotte, l'overhead introdotto dalla parallelizzazione potrebbe effettivamente superare i vantaggi derivanti da tale approccio. Pertanto, è fondamentale bilanciare attentamente queste due considerazioni per ottimizzare le prestazioni del sistema. È interessante osservare che, in alcuni casi, il throughput è notevolmente elevato, superando di gran lunga le previsioni basate sui MB/s misurati. Questo fenomeno può essere attribuito con certezza a meccanismi di caching implementati da FUSE e/o NFS. Tali meccanismi, essendo inevitabili, possono influenzare significativamente le prestazioni del sistema. Inoltre si osserva che l'utilizzo della CPU aumenta con l'incremento della dimensione dei blocchi quando si utilizza TCFS. Parallelamente, anche la velocità di lettura e scrittura tende ad aumentare. Pertanto, la scelta della dimensione dei blocchi è un aspetto cruciale. Su sistemi con prestazioni inferiori, potrebbe essere preferibile optare per blocchi di dimensioni più piccole, anche se ciò potrebbe comportare una riduzione della velocità. Al contrario, su sistemi più performanti, si potrebbe considerare l'uso di blocchi di dimensioni maggiori.

5.7 Possibili attacchi e contromisure

Di seguito si presenta una lista dei possibili attacchi a TCFS:

AES in modalità CBC (Cipher Block Chaining)

- **Padding Oracle Attack**: Questo attacco sfrutta il modo in cui alcuni sistemi rivelano se il padding di un messaggio crittografato è valido o meno[12]. Un attaccante può usare queste informazioni per decifrare il messaggio byte per byte.[7]
- **IV Reuse**: Se l'IV viene riutilizzato con la stessa chiave, un attaccante può lanciare vari attacchi, tra cui attacchi di tipo chosen-plaintext. [6]

AES in modalità GCM (Galois/Counter Mode)

- **Forgery Attacks**: Se si riutilizza lo stesso IV per ogni crittografia eseguita con la stessa chiave, l'applicazione diventa vulnerabile agli attacchi di falsificazione. [14]
- **Ferguson and Saarinen Attacks**: Questi attacchi sfruttano le debolezze nella generazione del tag di autenticazione in GCM. [13]

AES in modalità CTR (Counter Mode)

- **IV Reuse**: Come per la modalità CBC, il riutilizzo dell'IV può portare a vari attacchi, tra cui attacchi di tipo chosen-plaintext. [3]
- **Malleability**: In modalità CTR, un attaccante può modificare i bit del testo cifrato senza conoscere la chiave, alterando così il testo in chiaro. [3]

AES senza IV

L'uso di AES senza un IV può portare a vari problemi. Ad esempio, se si crittografa lo stesso messaggio più volte con la stessa chiave, il testo cifrato sarà lo stesso ogni volta. Questo può rivelare informazioni all'attaccante. [2] Inoltre, AES è resistente agli attacchi di tipo known-plaintext solo se si utilizza un IV diverso e casuale per ogni singolo messaggio. [11]

5.8 Valutazioni personali

A mio avviso, questa "proof of concept" ha superato le aspettative, dimostrando la sua efficacia. In un contesto di utilizzo reale, l'adozione di TCFS non sembra comportare una riduzione delle prestazioni tale da essere rilevante per l'utente. Questo suggerisce che TCFS può essere implementato con successo senza compromettere l'esperienza dell'utente.

Capitolo 6

Conclusioni

6.1 Limiti e prospettive future del lavoro

Sebbene la versione attuale di TCFS funzioni nel quadro complessivo, non possiamo ignorare gli aspetti che sono rimasti incompleti o che funzionano solo parzialmente. L'algoritmo di cifratura AES GCM incontra notevoli difficoltà nel gestire blocchi di dimensioni superiori a 6 KB. Si è osservato che l'utilizzo della memoria aumenta drasticamente con l'avanzare del programma, nonostante la memoria residente di TCFS e quella virtuale rimangano invariate. È probabile che il problema sia la frammentazione della memoria dovuta a uno o più dei seguenti fattori:

1. **libssl**: Gestisce la memoria in modo particolare, cioè mantiene una lista interna di spazi di memoria liberi, invece di chiamare direttamente le funzioni malloc e free, il che può portare a una frammentazione interna della memoria. Questo perché la memoria libera può essere sparpagliata in diverse posizioni in porzioni piccole e non continue.

2. **OS e compilatore**: Se il sistema operativo o il compilatore non liberano la memoria al momento dell'invocazione della funzione 'free' può verificarsi la frammentazione esterna. Questo accade quando la memoria libera è divisa in blocchi non contigui, impedendo l'allocazione di nuovi processi che richiedono blocchi di memoria più grandi.

3. **Lazy memory allocation**: Questa tecnica, utilizzata per ritardare l'allocazione della memoria fino a quando non è effettivamente necessaria, può anch'essa contribuire alla frammentazione. Se la memoria non viene allocata immediatamente quando vengono richiesti numerosi byte, ciò può portare a rallentamenti e potenzialmente a frammentazione se le richieste di memoria successive richiedono blocchi di memoria contigui.

Anche AES CTR può effettivamente riscontrare problemi quando si tratta di gestire blocchi di dimensioni considerevoli, in particolare quelli di dimensione pari o superiore a 32 MB. Questo è comprensibile, considerando che AES è un cifrario a blocchi con una dimensione di blocco fissa di 128 bit (16 byte). Anche se 32 MB possono sembrare una dimensione di blocco ragionevole, possono essere sufficientemente grandi da causare problemi di gestione della memoria, specialmente se la memoria non viene liberata in modo efficiente o se ci sono problemi di frammentazione della memoria. Tuttavia, è importante sottolineare che non

esiste un motivo intrinseco per cui il plaintext debba essere un multiplo della dimensione del blocco del cifrario. Infatti, AES CTR esegue l'operazione XOR con il plaintext per ciascun blocco di output del cifrario. Ciò implica che il plaintext può avere qualsiasi lunghezza, e non necessariamente un multiplo della dimensione del blocco. Di conseguenza, AES CTR dovrebbe essere in grado di gestire messaggi di qualsiasi lunghezza. In conclusione, sebbene AES CTR possa riscontrare problemi con blocchi di 32 MB, ciò non dovrebbe limitare la sua utilità in molti contesti. Potrebbe essere necessario apportare ulteriori ottimizzazioni o considerare alternative se la dimensione del blocco diventa un problema.

Un altro problema significativo riguarda la gestione approssimativa degli IV. L'idea è di utilizzare un IV per la cifratura dei file e di utilizzare un IV diverso per ogni blocco del file. Tuttavia, l'IV è gestito correttamente solo per la cifratura della chiave del file. Questo ci porta al problema successivo: sarebbe necessario integrare una gestione della chiave dei file più sicura, simile a come è stata sviluppata la libreria key-utils. Questa gestione della chiave dei file non dovrebbe permettere di scrivere la chiave in un memory dump e dovrebbe sovrascrivere la chiave con 0 in memoria dopo l'utilizzo. Altre parti del progetto rimangono incomplete, il threshold sharing è stato implementato parzialmente, non è possibile recuperare lo share ottenuto da Shamir e non è stata implementata una utility per l'utente, in modo da agevolare l'utilizzo del FS. Inoltre sarebbe ideale creare un pacchetto per il FS e fornire in esso i binding per il comando mount in modo da poter utilizzare "mount.tcfs" per utilizzare il FS.

Ci si propone di risolvere e implementare tutti i cambiamenti necessari e implementare tutte le funzionalità mancanti in modo da rendere TCFS un FS completo e utilizzabile da tutti.

Capitolo 7

Glossario

FS: Filesystem

OS: Sistema Operativo

FUSE: Filesystem in USerspacE

API: Advanced Programming Interface

ID: IDentificatore

IO: Input Output

TAG: Termine inglese che rappresenta la parola "etichetta"

TCFS: Transparent Cryptographic FileSystem

Bibliografia

[1] Cybersecurity 360. Privacy e dati: una questione di consapevolezza e gestione rischi e sicurezza, s.d.

[2] Ismail Negabi Smail Ait El Asri Samir El Adib and Naoufal Raissouni. Deep learning-based power analysis attack for extracting aes keys on atmega328p microcontroller, 2023.

[3] Rasheed Ahmad Izzat Alsmadi Wasim Alhamdani and Lo'ai Tawalbeh. Zero-day attack detection: a systematic literature review, 2023.

[4] Altalex. Protezione dei diritti nell'era digitale: tratti essenziali e capisaldi normativi, 2020.

[5] Altalex. Ruolo data acquisizioni big tech, 2020.

[6] Nerd Cred. Aes cbc mode – chosen plaintext attack, 2021.

[7] Debranjan Pal Md Rasid Ali Abhijit Das and Dipanwita Roy Chowdhury. A cluster-based practical key recovery attack on reduced-round aes using impossible-differential cryptanalysis, 2022.

[8] Charles P. Wright Jay Dave and Stony Brook University Erez Zadok. Cryptographic file systems performance: What you don't know can hurt you.

[9] Charles P. Wright Jay Dave and Stony Brook University Erez Zadok. Evaluation.

[10] 4C Legal. Fenomeno big data: protezione dei dati personali, s.d.

[11] Loïc Masure and Rémi Strullu. Side-channel analysis against anssi's protected aes implementation on arm: end-to-end attacks with multi-task learning, 2023.

[12] Sunday Adeola Ajagbe Oluwashola David Adeniji Adedayo Amos Olayiwola and Seun Femi Abiona. Advanced encryption standard (aes)-based text encryption for near field communication (nfc) using huffman compression, 2024.

[13] Markku-Juhani Olavi Saarinen. Cycling attacks on gcm, ghash and other polynomial macs and hashes, 2012.

[14] Hanno Böck Aaron Zauner Sean Devlin Juraj Somorovsky and Philipp Jovanovic. Nonce-disrespecting adversaries: Practical forgery attacks on gcm in tls.

Nel profondo del mio cuore, sento un'immensa gratitudine che non riesco a esprimere a parole. Questo viaggio, lungo e difficile, non sarebbe stato possibile senza l'amore e il sostegno di coloro che mi hanno accompagnato lungo la strada.

Ai miei genitori, che mi hanno sostenuto fin dai miei primi passi, non posso esprimere quanto sia grato. Il vostro amore incondizionato e il vostro sostegno costante sono stati la mia roccia. Non avrei potuto farcela senza di voi.

A mia sorella, grazie per essere sempre lì per me. La tua presenza è stata una fonte di conforto e forza nei momenti di dubbio e incertezza.

Alle mie cugine, che mi hanno guidato con saggezza e mi hanno regalato momenti di gioia inestimabile, grazie. I vostri consigli e la vostra compagnia hanno illuminato il mio cammino, non solo in questi anni.

Ai miei nonni e ai miei zii, la vostra generosità, sia morale che talvolta economica, mi ha permesso di godere di momenti di svago e divertimento che altrimenti non avrei avuto. Il vostro sostegno mi ha dato la forza di andare avanti.

A Matteo O., un amico che è diventato un fratello. La tua presenza costante dalla mia infanzia fino ad oggi è un dono inestimabile. Dalle nostre avventure su redM alla nostra impresa imprenditoriale nella vendita di cuscini in fibra di bamboo su healthysleep.it®, ogni momento condiviso con te è stato unico e prezioso. Le nostre idee, per quanto stravaganti, sono il riflesso della nostra amicizia unica e irripetibile. Non credo che incontrerò mai un'altra persona come te, perché sei un diamante raro in un mondo di pietre comuni. La tua amicizia è un tesoro che custodirò per sempre nel mio cuore. Grazie, Matteo, per essere una parte così importante della mia vita. La tua presenza ha arricchito la mia esistenza in modi che non avrei mai immaginato. Grazie, da tutto me stesso.

Questo viaggio è stato reso possibile grazie a tutti voi. Con tutto il mio cuore, vi ringrazio.

Riprendendo il discorso svago, uscite, bar e bevute, e qui i familiari vari possono anche smettere di leggere. No, seriamente, mamma, puoi smettere di leggere, non è necessario che continui... Un caloroso ringraziamento a Mattia, un vero e proprio campione olimpico del bere, un compagno di avventure notturne ineguagliabile, e un maestro indiscusso nella caccia ai Pokémon. Ma oltre a tutto questo, Mattia ha scelto di donarmi un'amicizia sincera che ha arricchito questi anni. La tua amicizia è un tesoro che custodisco con gratitudine nel mio cuore.

Un ringraziamento speciale va a Daniele R., il mio personale "Google degli esami". Senza di lui, probabilmente starei ancora cercando di capire come si fa un esame.

Vorrei ringraziare Valeria, la mia fonte personale di sorrisi. La tua amicizia e il tuo impegno in tutto ciò che fai sono una continua fonte di ispirazione.

Ringrazio Asia per il suo affetto genuino. Non posso esprimere a parole quanto sia importante per me.

E poi, un ringraziamento speciale va ancora una volta a Valeria e Asia, le mie coinquiline. Scusate se non pulivo spesso la cucina, giuro che avevo intenzione di pulire, presto o tardi... forse più tardi che prima.

A Mariantonietta, mia amica in questi anni. Con te ho condiviso ogni giorno, gioia, dolore, gossip, e tutto il resto. Mi chiedo ancora come fai a sopportarmi, ma so che lo fai con amore.

Grazie per essere sempre lì.

A Daniele F., il mio primo amico nel mio percorso universitario. Quando tutto sembrava andare male e tutti eravamo chiusi in casa, sei stato un faro di speranza. Grazie per essere entrato nella mia vita.

E a Francesco G., grazie per avermi mostrato che la mensa può essere un luogo di delizie culinarie. Scherzi a pare la tua genuinità e il tuo carisma sono un dono raro. Sei un insegnamento prezioso e un amico ancora più prezioso.

A Luca, ho ancora una foglia di alloro della tua corona di laurea nella cover del telefono. È il mio porta fortuna.

A tutti gli amici, vecchi e nuovi, che hanno arricchito la mia vita in modi innumerevoli, vorrei esprimere la mia più profonda gratitudine. Grazia, Yuri, Domenico I., Matteo C. E., Melissa, Emanuele, Camilla, Annarita, Denise, Domenico B., grazie al gruppo della mensa, grazie, ognuno di voi ha lasciato un'impronta indelebile nel mio cuore.

A tutti voi, ragazzi e ragazze, vi voglio bene. Questi anni non sarebbero stati gli stessi senza di voi. Grazie di cuore.

Vorrei ringraziare il prof. Cattaneo per l'impegno che ci ha messo nel seguirmi in questo percorso e per avermi proposto un progetto di tesi appassionante, su un argomento che rispecchia a pieno i miei interessi.

Se ho dimenticato di ringraziare qualcuno, vi prego di perdonarmi. Non è che ho la memoria di un pesce rosso, è solo che questi mesi sono stati più stancanti di un maratona di danza di 48 ore con le scarpe da ginnastica al contrario! Immaginatevi di correre su una pista di atletica con un paio di zoccoli olandesi, ecco, così mi sono sentito! Quindi, se il tuo nome non è stato menzionato, non preoccuparti, non è che ti ho dimenticato. È solo che il mio cervello è attualmente più intricato di un labirinto di spaghetti al dente, costruito da un gruppo di pinguini in smoking, all'interno di un convento ortodosso olandese, dove suore pallavoliste giocano a badminton con racchette in fibra di santità concentrata, mentre un koala su un mono-ciclo giocola con ananas e un coro di rane austriache canta l'inno francese in greco antico! Quindi, se il tuo nome non è apparso in questa lista, non preoccuparti, non è un segno che ti ho dimenticato. È più un segno che ho bisogno di un bel sonno...! Grazie per la comprensione.

"Adatta te stesso alle cose a cui la sorte ti ha assegnato. E ama, ma veramente,
gli uomini coi quali il destino ti ha unito."
- Marco Aurelio